



LEYENDA

Documentación técnica completa del proyecto

Versión: 1.3.1 — publicada el 22 de septiembre de 2026

Alcance: Cada fórmula, cada constante y dónde vive cada pieza en el código

Motor: HTML / CSS / JavaScript vanilla, sin dependencias ni build

Leyenda

Simulador de carrera de un futbolista, de principiante a leyenda (o al fracaso). Aplicación web de una sola página (SPA), sin backend ni base de datos externa: todo el motor corre en el navegador, en **Vue 3 + TypeScript + Pinia + Vite**.

Versión: 1.3.1 — publicada el 22 de septiembre de 2026 · 14:54.

Este documento describe **absolutamente toda la lógica del juego**: cada fórmula, cada constante de balance y dónde vive cada pieza en el código. Está pensado como referencia técnica completa, no como introducción rápida — si buscas "cómo se juega" en términos de jugador, ver el *Manual de Juego* aparte.

Nota de historia: el proyecto arrancó como un sitio 100% vanilla (HTML/CSS/JS sin build, sin módulos). Esa versión fue reescrita por completo a Vue 3 + TypeScript + Pinia — un *port* fiel motor por motor, no un rediseño — y, una vez validada en producción, el código vanilla se eliminó del repositorio. Este documento describe la versión Vue actual, que es la única que existe hoy.

Índice

1. Cómo correr el proyecto
2. Estructura de archivos
3. Flujo general del juego
4. Creación de personaje
5. Elección de club inicial
6. El motor de carrera — visión general
7. Calendario de temporada
8. El ciclo de un tramo, paso a paso
9. Participación: cuántos partidos juegas
10. Estadísticas del tramo: goles, asistencias, MVP, rating
11. Progresión de OVR
12. Estado de forma
13. Lesiones
14. Sistema de competiciones (liga, copas, clasificación internacional)
15. Valor de mercado

16. Sistema de fichajes y ofertas
 17. Fin de carrera: retiro y resumen
 18. Solicitud de cambio de dorsal
 19. Selección nacional
 20. Banco de eventos de temporada
 21. Base de datos de ligas y equipos
 22. Interfaz: componentes, composables y responsive
 23. Persistencia y estado
 24. Tabla completa de constantes de balance
 25. Limitaciones conocidas y notas para el futuro
-

1. Cómo correr el proyecto

Todo el código vive bajo `app/` — una SPA Vue armada con Vite. Necesita Node instalado:

```
cd app
npm install
npm run dev
```

Levanta el servidor de desarrollo de Vite (con recarga en caliente) en `http://localhost:5173`. La configuración ya está en `.claude/launch.json` para levantarlo automáticamente.

Otros scripts de `app/package.json`:

Comando	Qué hace
<code>npm run build</code>	Type-check (<code>vue-tsc</code>) + build de producción (<code>vite build</code>) a <code>app/dist/</code>
<code>npm run preview</code>	Sirve el build de producción ya generado, para probarlo localmente
<code>npm run test:unit</code>	Corre la suite de tests (Vitest)
<code>npm run type-check</code>	Solo el chequeo de tipos, sin build
<code>npm run lint</code>	<code>oxlint</code> + <code>eslint</code> , con <code>--fix</code>
<code>npm run format</code>	Prettier sobre <code>app/src/</code>

2. Estructura de archivos

Leyenda/	
├─ README.md	Este documento
├─ Leyenda - Documentacion Tecnica.pdf	Copia en PDF de este documento
├─ Leyenda - Manual de Juego.pdf	Copia en PDF del manual de juego
├─ netlify.toml	Config de deploy (build desde app/, publica app/dist)
├─ dev/	
│ └─ pdf_build/	Pipeline para regenerar los dos PDF de arriba a partir de este README y de manual.md (marked + wrap + puppeteer)
└─ app/	La SPA - todo el juego vive acá
│ └─ index.html	Punto de entrada de Vite
│ └─ package.json	
│ └─ public/	
│ │ └─ _redirects	Fallback de SPA para Netlify (todas las rutas → index.html)
│ │ └─ assets/	
│ │ │ └─ logo/	Logo del juego
│ │ │ └─ escudos/	
│ │ │ │ └─ equipos/	Escudos reales de cada club (PNG)
│ │ │ │ └─ ligas/	Escudos/logos de cada liga
│ │ │ │ └─ trofeos/	Siluetas de trofeos reales (se pintan de dorado vía CSS)
mask)	
└─ src/	
│ └─ App.vue / main.ts	Raíz de la app y punto de arranque
│ └─ router/index.ts	Las 3 rutas del juego (ver más abajo)
│ └─ assets/base.css	Tokens de diseño globales + reset, compartidos por todo
│ └─ stores/career.ts	El motor del juego: estado + simulación de la carrera
(Pinia)	
└─ game/	
│ └─ config.ts	GameConfig - TODAS las fórmulas y constantes de balance
│ └─ career-types.ts	Tipos del estado de una carrera (Temporada, Player, etc.)
│ └─ format.ts	Formato/presentación puros (ovrTierColor, valor de
mercado)	
└─ ovr-chart.ts	Matemática del gráfico de evolución de OVR (sección 17)
└─ resumen-canvas.ts	Tarjeta para compartir el resumen de carrera (canvas)
└─ player-draft.ts	Borrador de jugador (entre creación de personaje y club)
└─ initial-offers.ts	Ofertas de club inicial (portado de equipo.js)
data/	
└─ database.ts	GameDatabase - ligas, equipos, competiciones y
selecciones	
└─ database-helpers.ts	Lookups compartidos (equipoDe, ligaDe)
└─ events.ts	GameEvents - banco de 226 eventos de decisión + lesiones
└─ countries.ts	Los 47 países de la creación de personaje
└─ composables/	
│ └─ useAnimatedNumber.ts	Interpolación de números/anillo (ease-out cúbico, 900ms)
│ └─ useToast.ts	Sistema de notificaciones cortas
│ └─ resetZoom.ts	Resetea el zoom de iOS al navegar, cerrar un modal o
cerrar el teclado	
└─ components/	

```

app)
├── CrestImg.vue / FlagImg.vue    Escudo/bandera con fallback (compartidos por toda la
├── carrera/                      Componentes específicos de la pantalla de carrera:
│   ├── HeroPanel, SpotlightCard, TimelineList, DecisionsPanel, DecisionCardItem,
│   ├── OfertaCardItem, LesionCardItem, NumeroModal, ResumenModal, TrofeoIcon, TrophyBadges
├── views/
│   ├── PersonajeView.vue        Pantalla 1: creación de personaje
│   ├── EquipoView.vue           Pantalla 2: elección de club inicial
│   └── CarreraView.vue          Pantalla 3: el juego en sí (pantalla principal)

```

Rutas (`router/index.ts`) — reemplazan a las 3 páginas HTML de la versión original:

- `/` → `PersonajeView.vue` (creación de personaje)
- `/equipo` → `EquipoView.vue` (elección de club inicial)
- `/carrera` → `CarreraView.vue` (el juego)

Todo el estado/lógica de balance vive en `GameConfig` (`game/config.ts`) y `GameDatabase` / `GameEvents` (`data/`), igual que antes — pero ahora como módulos ES importados donde hacen falta, no objetos globales cargados por `<script>` . Vite se encarga del bundling; no hay archivos sueltos que cargar en un orden particular.

3. Flujo general del juego

```
/ - PersonajeView (crear personaje)
|   guarda en localStorage["leyendaPlayerDraft"]:
|   { apellido, numero, pierna, edad, pais, flag, paisCode, posicion }
▼
/equipo - EquipoView (elegir 1 de 4 ofertas de club inicial)
|   agrega al mismo borrador: { equipoId, ovrInicial }
▼
/carrera - CarreraView (el juego)
|   arranca la Temporada 1 con ese club y ese OVR inicial
|
|   ┌───────────────────────────────────────────────────────────┐
|   │ Se repite temporada tras temporada:                    │
|   │ calendario de 4 pausas → tramos → cierre                │
|   └───────────────────────────────────────────────────────────┘
|
▼
Retiro (forzoso por edad, o elegido) → resumen de carrera → volver a /
```

A diferencia de cómo arrancó el proyecto, **la carrera sí se guarda**: cada acción que cambia el estado la persiste en `localStorage["leyenda-carrera"]`, así que cerrar la pestaña o recargar la página no pierde el progreso — ver [sección 23](#) para el detalle completo.

4. Creación de personaje (`views/PersonajeView.vue`)

Formulario de 3 pasos (acordeón en móvil, los 3 siempre abiertos en escritorio):

Paso	Campo	Detalle
1. ¿Quién eres?	Apellido	Texto libre, máx. 16 caracteres, se muestra en mayúsculas en la camiseta.
	Edad	Botones de 16 a 19 años (<code>GameConfig.EDAD_MIN / EDAD_MAX</code> , <code>config.ts:393-394</code>).
	Pierna hábil	Izquierda / derecha — no afecta ninguna fórmula del juego , es solo cosmético (se guarda pero no se lee en ningún cálculo).
2. ¿De dónde eres?	País	47 países (<code>COUNTRIES</code> , <code>data/countries.ts:12</code>), con buscador. Define bandera y, en el paso siguiente, el pool de clubes iniciales.
3. ¿Dónde juegas?	Posición	12 posiciones sobre una cancha (<code>FILAS_CANCHA</code> , <code>PersonajeView.vue:31</code>): POR, DFC, LI, LD, MCD, MC, MI, MD, MCO, EI, ED, DC.

El **dorsal** se asigna solo al azar, no se elige — pero no parejo entre 1 y 99 (así, un debutante tenía la misma chance de arrancar con el 7 que con el 87, nada realista). `sortearDorsalInicial()` (`config.ts:406`, llamado desde `PersonajeView.vue:59`) sortea por bandas: **70%** de las carreras arranca con un número común (**1-30**), **20%** con uno menos común (**31-50**), y solo el **10%** restante con uno alto (**51-99**) — el caso ocasional, no la norma. Recién se puede pedir cambiarlo al cerrar la primera temporada (ver [sección 18](#)).

Al completar los 3 pasos y tocar "Comenzar carrera", se guarda un borrador en `localStorage["leyendaPlayerDraft"]` (`savePlayerDraft` , `game/player-draft.ts:11`, llamado desde `PersonajeView.vue:165`):

```
{ "apellido": "...", "numero": 42, "pierna": "derecha", "edad": 17,
  "pais": "Argentina", "flag": "AR", "paisCode": "ar", "posicion": "DC" }
```

y se navega a `/equipo` . Es un borrador intermedio, no la carrera en sí — se completa con `equipoId / ovrInicial` en el paso siguiente y recién se consume al confirmar el club (ver [sección 23](#)).

5. Elección de club inicial (`views/EquipoView.vue`)

Se presentan **4 ofertas de club**, elegidas así (`generarOfertasInicialesParaJugador` , `game/initial-offers.ts:23`):

- **Si el país elegido tiene una liga propia** en la base de datos (**24 de los 47** países de la creación de personaje, desde Alemania/Argentina/España hasta Bolivia/Costa Rica/Paraguay/El Salvador — todas las que tienen `pais` cargado en `GameDatabase.ligas` , ver [sección 21](#)), las 4 ofertas salen de esa liga, en esta banda fija (`OFERTAS_INICIALES` , `config.ts:505-510`, aplicada por `generarOfertasIniciales` , `config.ts:595`):
 - 2 clubes **humildes** (mitad de abajo por poder, dentro de esa liga)
 - 1 club **consolidado** (entre el 50% y el 85% por poder)
 - 1 club **al azar**, de cualquier categoría (la única chance de arrancar en un club grande)
- **Si el país NO tiene liga propia** (los otros 23 — mayormente africanos, asiáticos y del resto de Europa que todavía no tienen liga doméstica cargada), arranca "de extranjero" en una de las **5 grandes ligas europeas** elegida al azar (Premier League, La Liga, Serie A, Bundesliga, Ligue 1 — `LIGAS_GRANDES_EUROPEAS` , `config.ts`), con una banda más floja y sin favores (`OFERTAS_INICIALES_EXTRANJERO` , `config.ts:512-517`, aplicada por `generarOfertasInicialesExtranjero` , `config.ts:601`): 3 clubes humildes + 1 consolidado, nunca uno grande.

"Humilde" / "consolidado" / "grande" son percentiles por poder **dentro de la liga elegida** (`categoriaEquipoEnLiga` , `config.ts:562` — ver [sección 16.6](#) para qué es el poder de un club), no una categoría fija guardada en cada club: humilde es la mitad de abajo, consolidado el 50-85%, grande el 15% de arriba (`CATEGORIA_EQUIPO_PERCENTIL` , `config.ts:503`).

El **OVR inicial** con el que arrancarías en cada club se calcula recién al elegirlo, con `calcularOvrInicial(equipo, liga)` (`config.ts:540`):

1. Se combina el poder del equipo y el de la liga con pesos fijos: **55% equipo + 45% liga** (`OVR_PESO_EQUIPO` / `OVR_PESO_LIGA` , `calidadPoderCombinada` , `config.ts:472-485`), normalizado a una escala 0..1.
2. Ese 0..1 se mapea al rango **50–65** (`OVR_INICIAL_MIN` / `MAX` , `config.ts:467-468`) — un novato, por definición, nunca arranca más alto que eso, sin importar cuán grande sea el club.
3. Se le suma una "suerte" aleatoria de hasta ± 4 puntos (`OVR_SUERTE_VARIACION` , `config.ts:476`) y se redondea, recortado siempre entre 50 y 65.

El jugador nunca ve estos números crudos: en la tarjeta de oferta solo se muestra el nombre del club/liga, su escudo y su bandera — sin ninguna etiqueta de "qué tan grande es", a diferencia de versiones anteriores.

Al elegir un club se completa el borrador de `localStorage["leyendaPlayerDraft"]` con `equipoId / ovrInicial` (ver [sección 4](#)) y se navega a `/carrera`, donde `iniciarCarrera` (ver [sección 23](#)) consume ese borrador y lo borra — sin ningún toast de confirmación de por medio ("Fichaste por X, OVR inicial: Y") — la propia transición de pantalla ya comunica que la elección se hizo, y la sección 4 (creación de personaje) es la única que sí sigue mostrando un toast de bienvenida.

6. El motor de carrera — visión general (`stores/career.ts`)

Es un store de Pinia (`useCareerStore` , `career.ts:111`, 1200+ líneas): concentra el **estado del juego** y **toda la simulación**, pero a propósito **no** el render — eso vive en los componentes de `components/carrera/` / `views/CarreraView.vue` (ver [sección 22](#)), que reaccionan solos a los cambios de este estado en vez de que el motor tenga que "pintar" nada él mismo. Es la separación de capas que la versión original no tenía.

Estado central

```
const player = ref<Player | null>(null)
const temporadaActual = ref<Temporada | null>(null) // la temporada en curso
const temporadasFinalizadas = ref<Temporada[]>([]) // historial completo, una fila por
temporada
const temporadasEnClubActual = ref(0) // temporadas completas en el club
actual (período de gracia)
const carreraFinalizada = ref(false)
const edadRetiroForzoso = ref(0) // se sortea (41-45) al arrancar la
carrera, en iniciarCarrera()
```

`temporadaActual` (creada por `crearTemporada()` , `career.ts:145`, con su forma completa tipada en `Temporada` , `career-types.ts:117`) contiene, entre otras cosas: `numero` , `anio` , `equipoId` , `clubDuenoId` (no-null solo si estás a préstamo — ver [sección 16.9](#)), `ovr` , `partidos/goles/asistencias/mvp/sumaRating/promedio` , `valorMercado` , `trofeos[]` , `forma` , `titular` , `progreso` (0-100%), `calendario[]` , `checkpointIndex` , `tramoIndex` , `lesionActiva` , `bufferRendimiento` / `bufferEquipo` (efecto acumulado de las decisiones del tramo en curso, sin aplicar todavía), `competiciones` (liga + copa nacional + copa internacional de esa temporada) y todo lo de la selección nacional de esa temporada — `seleccion` , `tipoAnoSeleccion` , `convocatoriaPausa` , `seleccionPartidos` / `seleccionGoles` (ver [sección 19](#)).

`/carrera` está protegida por el router: si no hay una carrera activa en memoria ni guardada en `localStorage` (ver [sección 23](#)), redirige a `/` en vez de arrancar nada — ya no existe el fallback de "carrera demo" que tenía la versión original para poder abrir esa pantalla directo sin pasar por la creación de personaje.

La edad sube 1 año por cada temporada, nunca dentro de una misma temporada —
`getEdadActual()` (`career.ts:139`):

```
edadActual = edad de creación + (número de temporada actual - 1)
```

Todas las fórmulas que dependen de la edad (crecimiento/declive de OVR, riesgo de lesión, ventanas de fichaje, retiro) usan este valor.

7. Calendario de temporada

Cada temporada tiene 3 **"tramos"** (`TOTAL_TRAMOS_TEMPORADA` , `config.ts:1060`) — bloques de partidos que se simulan de una vez — separados por pausas donde el jugador interactúa: 3 de decisiones y, a partir de la Temporada 2, **1 sola ventana de fichajes**.

`crearCalendarioTemporada(numeroTemporada)` (`config.ts:1065`) arma:

Pausa	Momento (<code>progreso</code> %)	Nota
Oferta (pretemporada)	0%	Solo a partir de la Temporada 2 — en la 1 ya elegiste equipo en la creación, y en el resto es la única ventana de fichajes del año (ver nota abajo).
Decisión	aleatorio entre 5% y 45%	"antes de la mitad"
Decisión	aleatorio entre 0% y 100%	"en cualquier punto"
Decisión	aleatorio entre 92% y 99%	"último momento"

Las 4 (o 3, en la Temporada 1) se ordenan por este `progreso` para que el calendario quede cronológico **al narrar la temporada** — pero ese número es solo para ordenar pausas, no es lo que se muestra en pantalla (ver más abajo). Cada pausa de decisión resuelta dispara la simulación del tramo siguiente (`simularTramoYAvanzar`) y avanza al próximo checkpoint (`avanzarCheckpoint`); al agotarse el calendario, se cierra la temporada (`finalizarTemporada`).

El progreso que ves en pantalla (anillo/barra) es otro número: `temporadaActual.progreso` se recalcula en cada tramo como `partidosJugados` / `partidosMinimos` de la liga (`simularTramoYAvanzar` , `career.ts:609`), no a partir de la tabla de arriba — antes reusaba esos valores aleatorios de la tabla, así que la barra podía verse casi llena con 15 partidos jugados y saltar a más de 50% recién en el último tramo. Ahora sigue de cerca el avance real de partidos de liga (no puede ser 100% exacto porque la cantidad de partidos por tramo no es fija, pero da la ilusión correcta de avance).

Por qué una sola ventana, y siempre en pretemporada: antes había una segunda pausa de fichajes a mitad de año, lo que permitía que un traspaso partiera una temporada en dos (dos clubes distintos, dos filas de historial, el mismo año). Se sacó a propósito — ahora un fichaje (o un préstamo, ver [sección 16.9](#)) sale siempre con la temporada en cero (0 partidos jugados con el club nuevo) y la corres entera de punta a punta con ese club, tal como pasaría en la realidad con una ventana de pases real.

`resolveOferta` (`career.ts:1131`) no tiene ninguna rama de "traspaso a mitad de camino": cambiar de

club siempre resetea `competiciones` (liga + copa nacional; la clasificación internacional NO se hereda, es del club, no del jugador) desde cero.

Alto Impacto: al crear la temporada se sortea si va a haber un evento de alto impacto (30% de probabilidad) y, si sale, en cuál de los 3 tramos va a aparecer (`altoImpactoPausa` , `career.ts:145`, dentro de `crearTemporada`). Ver [sección 20](#).

8. El ciclo de un tramo, paso a paso

Todo pasa en `simularTramoYAvanzar()` (`career.ts:609`), disparado al resolver la última decisión pendiente de una pausa:

1. **Partidos del club esta franja**, sumando las 3 competiciones activas:
 - Liga: `partidosLigaParaTramo` reparte el total de la temporada entre los 3 tramos a partes iguales, y el último tramo absorbe el resto del redondeo.
 - Copa nacional / internacional: `resolverKnockout` — en su tramo de "partidos mínimos" juega la ronda garantizada; después, un tramo por cada ronda extra disponible, con una tirada de `probAvanzarRonda(fuerza)` para seguir viva (si pierde, queda eliminado para el resto de la temporada).
2. **Titularidad de este tramo**: `calcularTitular(pesoTitular, ovr, rendimientoAcumulado)` — una tirada (ver fórmula en [sección 9](#)) decidida **antes** de calcular cuánto juegas, para que ser titular realmente sume participación (no es solo un badge decorativo). Al cierre del tramo, el resultado también ajusta `pesoTitular` de cara al próximo (paso 7 más abajo).
3. **Cuántos de esos partidos juegas tú** (no todos, ver [sección 9](#)) — si estás lesionado, **cero**, sin excepción.
4. **Resultado del tramo** (`GameConfig.simularTramo` , ver [sección 10](#)): goles, asistencias, MVPs y la suma de ratings de esos partidos.
5. Se acumulan a las estadísticas de la temporada, se recalcula el promedio (`sumaRating / partidos`).
6. **Se ajusta el OVR** (`ajustarOvrTramo` , [sección 11](#)) y se recalcula el **valor de mercado** con el nuevo OVR.
7. **Se ajusta `pesoTitular`** con el rating de este tramo (`ajustarPesoTitular` , ver [sección 9](#)) — de cara al próximo tramo, no a este que ya se jugó.
8. Se descuenta un tramo a la lesión activa, si había una; al llegar a 0, se da de alta.
9. Se avanza `tramoIndex` y se recalcula el `progreso` mostrado en pantalla a partir de `partidosJugados / partidosMinimos` de la liga (ver [sección 7](#)).
10. Se anima el spotlight (anillo de progreso + contadores) y, 1050ms después (`ANIMACION_TRAMO_MS` + margen, `config.ts:1061`), se pasa a la próxima pausa (o se cierra la temporada si no queda ninguna) — la demora es a propósito: si el siguiente checkpoint avanzara al toque, la animación quedaría cortada a mitad de camino.

Al **cerrar la temporada** (`finalizarTemporada` , `career.ts:866`):

- Se tira si ganas la **liga** y, si tu copa nacional llegó a la final, si la ganas también (fórmulas en [sección 14](#)).
- Se decide la **clasificación internacional de la próxima temporada** según qué tan bien te fue.

- Se archiva la temporada en `temporadasFinalizadas`, se crea la siguiente (heredando OVR y club), y se habilita el pedido de cambio de dorsal.
-

9. Participación: cuántos partidos juegas

Los partidos de arriba son los del **equipo**; cuántos de esos juegas tú depende de tu nivel, tu momento y si eres titular ese tramo.

Titularidad — ya no es un sorteo nuevo e independiente cada tramo: hay un valor persistente **pesoTitular** (0-1, "cuánto te ganaste el puesto DE VERDAD") que se arrastra tramo a tramo e incluso de una temporada a la siguiente mientras sigas en el mismo club. Una gran temporada ya no se "olvida" al arrancar la próxima, y un jugador titular indiscutido no vuelve a ser una moneda al aire de un día para el otro.

Arranca en **PESO_TITULAR_INICIAL = 0.4** (**config.ts:1455**, novato o recién fichado — hay que ganarse el puesto) y se resetea a ese mismo valor en cada transferencia (**resolveOferta** , **career.ts:1131**); si te quedas en el club, se hereda de una temporada a la siguiente sin tocar.

Después de cada tramo, **ajustarPesoTitular(pesoActual, ratingTramo, estabaLesionado)** (**config.ts:1467**) lo mueve según cómo te fue:

```
si estabas lesionado:      delta = PESO_TITULAR_CASTIGO_LESION      (-0.03)
si no jugaste ningún partido: delta = PESO_TITULAR_CASTIGO_SIN_MINUTOS (-0.05)
si no:                      delta = clamp((ratingTramo - 6.5) × 0.05, -0.08, 0.12)
pesoTitular = clamp(pesoActual + delta, 0.05, 0.95)
```

Y **calcularTitular(pesoTitular, ovr, rendimientoAcumulado, edad)** (**config.ts:1514**) decide el tramo actual:

```
prob = clamp(pesoTitular + (ovr - 55) × TITULAR_OVR_PESO + rendimientoAcumulado × 0.03 -
penalizaciónEdad, 0.08, 0.95)
(TITULAR_OVR_REFERENCIA = 55, TITULAR_OVR_PESO = 0.02)
```

pesoTitular sigue siendo el factor dominante — ya no decide todo un sorteo desde cero — pero el empuje del OVR se duplicó (antes 0.01): con el coeficiente viejo, un club podía seguir plantando de arranque a un novato de nivel bajo con bastante frecuencia con solo un **pesoTitular** mediano, porque el OVR casi no pesaba nada en la decisión — quedaba desalineado de cuánto sí pesa el nivel a la hora de cuántos partidos juegas en total (ver la fórmula de participación más abajo, que ya pesaba el OVR mucho más fuerte). Con 0.02 el efecto es real sin pasar a ser el factor dominante: **pesoTitular** sigue siendo lo que más importa.

penalizaciónEdad (**TITULAR_PENALIZACION_NOVATO_MAX = 0.25** , **config.ts:1510-1512**, mismo plateau 19→24 que la penalización de participación más abajo, con sus propias constantes — no comparte **TITULAR_EDAD_NOVATO_*** con **PARTICIPACION_EDAD_NOVATO_*** a propósito): sin esto, **calcularTitular** no sabía nada de tu edad — un novato de 17 con buen OVR de potencial (63,

techo típico de novato) y el `pesoTitular` inicial (0.4) ya tenía ~56% de chance de ser titular cada tramo, y si salía titular se llevaba además el `+0.2` de `PARTICIPACION_BONUS_TITULAR` en la fórmula de participación de abajo — ese bonus terminaba neutralizando buena parte de la penalización de edad que ya tiene la participación. Con la penalización, esa misma situación queda en ~31%.

La etiqueta que ves antes de jugar el primer tramo de una temporada nueva (`crearTemporada`, `career.ts:145`) ya no arranca fija en "Suplente" — se proyecta directo desde el `pesoTitular` heredado (`pesoTitular ≥ 0.5` → Titular). Antes quedaba en `false` a secas hasta que se simulaba el primer tramo, así que toda temporada nueva mostraba "Suplente" un instante, sin importar cuánto te hubieras ganado el puesto la temporada anterior — se leía como "una gran temporada no sirvió de nada".

Capitanía: ganarte el puesto no es solo cuestión de minutos — quedarte varias temporadas seguidas en el mismo club con `pesoTitular` alto de verdad te hace capitán. Se evalúa al cerrar cada temporada, antes de armar la siguiente (`finalizarTemporada`, `career.ts:983-999`):

```
esCapitan = temporadasEnClubActual ≥ CAPITAN_UMBRAL_TEMPORADAS (3)
            && pesoTitularProxima ≥ CAPITAN_UMBRAL_PESO_TITULAR (0.75)
```

(`config.ts:2033-2034`). `pesoTitularProxima` es el peso con el que arranca la PRÓXIMA temporada — el heredado si sigues en el mismo club, o `PESO_TITULAR_INICIAL` si vuelves de un préstamo (ver [sección 16.9](#)) — no el que acaba de cerrar, para que el chequeo use el valor real con el que arrancas. La primera vez que se cumple (transición de `false` a `true`) dispara un aviso puntual ("¡Te nombraron capitán de X!"); mientras se mantenga, no se repite.

Es un estado de la `Temporada` (`capitan: boolean`, `career-types.ts:137`), no de la carrera entera — se resetea a `false` en `resolveOferta` tanto en un traspaso real como en un préstamo (`career.ts:1147 / :1162`), igual que `pesoTitular`: la capitanía es de cancha, se re-gana en el club nuevo. Como `temporadasEnClubActual` ya resetea sola en un traspaso real (no en un préstamo, donde el reloj del club dueño sigue corriendo — ver [sección 16.9](#)), no hace falta ninguna lógica extra para que la capitanía se re-evalúe correctamente club por club. Se muestra como badge "Capitán" junto a Titular/Suplente en el spotlight, y como sufijo "(C)" junto al nombre del club en cada fila del historial donde aplicó (ver [sección 22](#)).

Probabilidad de jugar cada partido — `probabilidadJugar(ovr, rendimientoAcumulado, forma, esTitular, edad, promedioTemporada)` (`config.ts:1909`):

```
prob = 0.65 (PARTICIPACION_BASE)
      + (ovr - 55) × (ovr < 55 ? 0.08 : 0.02) (PARTICIPACION_OVR_REFERENCIA, _OVR_PESO_BAJ0,
      _OVR_PESO_ALTO)
      + rendimientoAcumulado × 0.03
      + bonusForma (ver tabla abajo)
      + 0.2 si eres titular este tramo (PARTICIPACION_BONUS_TITULAR)
      - penalización de novato (ver más abajo - 0 desde los 24 años)
      + (promedioTemporada - 6.5) × 0.15 (PARTICIPACION_PESO_RENDIMIENTO_REAL, null en el
      primer tramo)
recortado entre 0.15 (PARTICIPACION_MIN) y 0.92 (PARTICIPACION_MAX)
```

Forma	Bonus de participación
Inspirado	+0.15
En plenitud	+0.12
Animado	+0.06
Regular	0
Desanimado	-0.08
Bajo de forma	-0.15
Tocado físicamente (lesionado)	-0.35

El peso del OVR es **asimétrico a propósito**: por debajo de la referencia (55) castiga fuerte, por encima suma suave. Antes era un único coeficiente parejo de 0.01 — tan chico que un novato del piso real de OVR inicial (`OVR_INICIAL_MIN` = 50, ver [sección 5](#)) todavía terminaba jugando el **60%** de la temporada, nada de "suplente de verdad". Para castigar eso con un coeficiente único, el techo de los novatos buenos (`OVR_INICIAL_MAX` = 65) hubiera saturado directo al 100%, sin dejar margen para diferenciar a un jugador ya asentado. Con el peso partido en dos:

OVR	Antes (0.01 parejo)	Ahora
50 (piso real de novato)	60%	25%
55 (referencia)	65%	65% (sin cambio)
65 (techo de novato)	75%	85%
68 en adelante (jugador ya asentado)	~90%+	92% (techo — ni el mejor juega el 100% siempre, hay rotación/descanso)

Penalización por edad, para novatos de verdad (`PARTICIPACION_EDAD_NOVATO_PLATEAU_HASTA` = 19 / `_HASTA` = 24 , `PARTICIPACION_PENALIZACION_NOVATO_MAX` = 0.35): ni el OVR ni la fórmula de arriba distinguían la edad — un chico de 16 años con el OVR inicial típico (50-65, muy cerca del "neutral" de 55) ya arrancaba con 65-90% de probabilidad de jugar cada partido, sin importar si tenía 16 o 30 años. Ningún club le da la titularidad de entrada a un jugador de 16-19 solo porque su OVR no es terrible — se lo tiene que ganar con el tiempo. Es un **plateau, no un tapering desde el primer año**: se sostiene al máximo hasta los 19 (un club no empieza a confiar de a poco apenas cumples 17) y recién ahí empieza a bajar:

```

si edad ≤ 19: penalización = 0.35                                     (el máximo, sin excepciones)
si no:      progreso      = clamp((24 - edad) / (24 - 19), 0, 1)
              penalización = progreso × 0.35

```

De 16 a 19 resta siempre el máximo (-0.35); recién de 19 a 24 baja lineal hasta desaparecer del todo. Sola, esta penalización no alcanza: con un OVR de arranque/potencial apenas por encima del neutral (55) — la mayor parte del rango 50-65 — $(\text{ovr} - 55) \times 0.02$ ya cancelaba buena parte del -0.35, sin importar qué tan mal jugaras de verdad. Ahí es donde entra el peso del rendimiento real, justo abajo.

Peso del rendimiento REAL de la temporada ($\text{PARTICIPACION_PESO_RENDIMIENTO_REAL} = 0.15$, sobre promedioTemporada — el mismo promedio de rating acumulado que ya usa $\text{calcularFuerzaCampana}$, null en el primer tramo porque todavía no hay partidos): antes probabilidadJugar no se enteraba de tus goles/asistencias/rating reales, solo de tu OVR, tu forma y las decisiones de evento — dos jugadores con el mismo OVR jugaban igual de seguido sin importar si uno estaba rindiendo genial y el otro pésimo en cancha. Un tramo flojo de verdad ahora te resta minutos del tramo siguiente, y uno bueno te los suma. Arrancó en 0.06, pero ese peso era demasiado débil frente al término de OVR de arriba: un novato de OVR 60 con rendimiento parejo ($\text{promedioTemporada} = 6.5$) ya tenía ~40% de probabilidad de jugar cada partido por el solo hecho de su OVR de arranque, y aunque jugara realmente mal (promedio ~4.0) apenas bajaba a ~25% — suficiente para acumular más de 20 partidos entre los 16 y los 19 con números pobres, porque el OVR potencial seguía comprándole minutos que el rendimiento real no alcanzaba a descontar. Con 0.15, ese mismo novato (OVR 60, promedio ~4.0) cae directo al piso de participación (**15%**, ~5 partidos de 34), mientras que uno que sí rinde bien (promedio ~8.0) sube a ~63% (~21 partidos de 34) — el rendimiento real, no el OVR potencial de arranque, es ahora lo que de verdad decide tus minutos como novato.

$\text{partidosJugador} = \text{redondeoEstocastico}(\text{partidosClub} \times \text{prob})$ (ver "redondeo estocástico" en [sección 11](#)), recortado entre 0 y los partidos totales del club ese tramo. Si hay una lesión activa, partidosJugador es directamente 0, sin pasar por esta fórmula.

10. Estadísticas del tramo: goles, asistencias, MVP, rating

`GameConfig.simularTramo({ partidos, posicion, ovr, rendimientoAcumulado, fuerzaLiga, factorTalento })` (`config.ts:1218`) recorre partido por partido (de los que juegas tú, no los del equipo).

Propensión por posición individual

(`PROPENSION_GOL_POSICION` / `PROPENSION_ASISTENCIA_POSICION`, `config.ts:1108` y `:1114`): cada una de las 12 posiciones tiene su propio perfil — ya no comparten uno de 5 grupos compartidos como antes (ahí "ataque" mezclaba EI/ED/DC con el mismo número, así que un delantero centro rendía idéntico a un extremo; "medio" mezclaba los 5 mediocampistas por igual). Ahora:

Posición	Prob. de gol por partido (base)	Prob. de asistencia por partido (base)
Arquero (POR)	0.3%	0.3%
Central (DFC)	3%	2%
Lateral (LI/LD)	2%	9%
Mediocampista defensivo (MCD)	4%	8%
Mediocampista central (MC)	6%	11%
Mediocampista de banda (MI/MD)	6%	13%
Mediocampista ofensivo (MCO)	11%	15%
Extremo (EI/ED)	20%	9%
Delantero centro (DC)	27%	5%

El DC es el mayor goleador de la cancha — más que el extremo, que reparte más entre gol y asistencia — y dentro del mediocampo hay su propio orden: el MCD (el más contenedor) aporta menos que el MC, los de banda (MI/MD) asisten un poco más que el MC por los centros, y el MCO es el mediocampista más ofensivo, cerca del nivel de un extremo. `simularTramo` recibe la posición directa del jugador (con `MC` como respaldo neutral si no reconoce el valor); una posición **agrupada** en 5 categorías más amplias se sigue usando aparte, solo para el sorteo de candidatos a premios individuales — ver `GRUPOS_POSICION` en [sección 14.2](#).

Estas son las propensiones en el punto **neutral** de la curva de OVR de abajo (factor $\times 1$) — no en el piso de carrera. Una primera versión de la curva tenía ese punto neutral en $\sim$ OVR 65 (un jugador mediocre

rendía casi como uno "decente"), lo que dejaba estadísticas infladas — un delantero de 65 OVR llegaba a ~26 goles en 38 partidos. Recalibrado, el neutral pasó a ~OVR 75-78 (profesional sólido de verdad), pero eso dejó los primeros años de cualquier carrera (todo debutante arranca en 50-65 OVR, siempre por debajo de ese punto) con promedios de gol demasiado bajos para sentirse un jugador de verdad — un extremo de 60 OVR sacaba apenas ~2 goles en una temporada completa. Con el piso subido y el neutral corrido a ~OVR 72-73 (ver fórmula abajo), ese mismo debutante de 60 OVR ahora saca ~3.3 — y un DC del mismo OVR, con una propensión de gol 35% más alta (0.27 contra 0.20), saca proporcionalmente más todavía.

Factor de forma general del tramo:

```
factorOvr(ovr) = ESTADISTICAS_OVR_BASE + progreso^ESTADISTICAS_OVR_EXPONENTE × ESTADISTICAS_OVR_RANGO
                (progreso = (ovr - 45) / 54, recortado a 0-1; BASE=0.22, EXPONENTE=1.9, RANGO=2.78)
factorForma    = 1 + clamp(rendimientoAcumulado, -12, 12) × 0.05
factorLiga     = factorPorFuerzaLiga(ovr, fuerzaLiga)    (ver más abajo)
factorTalento  = talento oculto de la carrera (0.85x-1.2x, ver sección 11.2) - también pesa en las
                estadísticas, no solo en el crecimiento de OVR
factor         = max(0.3, factorOvr × factorForma × factorLiga × factorTalento)
```

Curva (exponente > 1), no una recta: en el OVR mínimo (45) el factor es 0.22, en el máximo (99) es 3.0 (el piso se subió de 0.15 a 0.22 y el exponente se suavizó de 2.2 a 1.9, sin tocar el techo — ver el porqué arriba). Con talento neutral (factorTalento = 1) y **factorLiga = 1**: un delantero de 60 OVR promedia ~3.3 goles en 34 partidos, uno de 65 ~4.6, uno de 75 (cerca del nuevo neutral) ~8.4, uno de 90 (estrella) ~17.6, y en el techo absoluto (99) ~26. El talento oculto agrega variación real sobre esos números a igual OVR: ese mismo delantero de 60 OVR saca ~2.7 goles con talento bajo (0.85x) o ~4.0 con talento alto (1.2x) — casi el doble de diferencia, para que no toda carrera se sienta "arranca mal, mejora con el tiempo": algunas ya se notan con algo especial desde el debut.

Competitividad de la liga (o selección) — sin esto, un jugador de 65 OVR rendía exactamente igual jugando en la liga de Colombia (fuerza ~55) que en la Premier League (fuerza ~96): el mismo OVR absoluto, sin importar contra qué nivel de rivales compite. **factorPorFuerzaLiga(ovr, fuerzaLiga)** (config.ts:1191) lo corrige:

```
referenciaLiga = OVR_CARRERA_MIN + (fuerzaLiga / 100) × (OVR_CARRERA_MAX - OVR_CARRERA_MIN)
ventaja        = ovr - referenciaLiga
factorLiga     = clamp(1 + ventaja × FACTOR_LIGA_COEFICIENTE, FACTOR_LIGA_MIN, FACTOR_LIGA_MAX)
                (COEFICIENTE = 0.024, MIN = 0.35, MAX = 2.2)
```

referenciaLiga mapea la fuerza de la liga (0-100) al mismo rango de OVR de carrera: una liga de fuerza 96 "espera" un nivel cercano al techo (~97), una de fuerza 55 espera algo bastante más modesto (~75). La diferencia entre tu OVR y esa referencia empuja el factor para arriba o para abajo. Con estos valores, un delantero de **70 OVR en Chile** (fuerza 60) promedia ~4 goles en una temporada de 34 partidos; ese mismo 70 OVR en la **Premier League** (fuerza 96) promedia ~2.1 — casi la mitad, con idéntico nivel absoluto. Se aplica también a los partidos con la selección nacional (con la fuerza de la selección rival, no la de tu liga de club — ver **sección 19**).

Goles por partido, sin techo real — `golesEnPartido(probGol)` (`config.ts:1208`), con `probGol = clamp(propensiónGol × factor, 0, 0.9)`:

```
si no sale (prob. 1 - probGol):      0 goles
si sale:                            1 gol
  con prob. probGol × 0.4 (PROB_SEGUNDO_GOL_FACTOR):  2 goles (doblete)
  con prob. probGol × 0.18 (PROB_TERCER_GOL_FACTOR):  3 goles (hat-trick)
```

Antes era un booleano puro (como mucho 1 gol por partido), así que la temporada entera jamás podía superar la cantidad de partidos jugados, por más crack que fueras — ahora un delantero de nivel alto puede perfectamente terminar una temporada con más goles que partidos. Las asistencias siguen siendo una tirada única por partido (`hizoAsistencia`, con `probAsistencia = clamp(propensiónAsistencia × factor, 0, 0.9)`).

MVP y rating del partido están conectados a cuántos goles/si hubo asistencia ESE partido puntual (no son sorteos independientes) — con un doblote o hat-trick, el bono escala con la cantidad de goles, no es todo-o-nada:

```
prob. de MVP = 0.09 × factor + golesPartido × 0.14 + 0.08 (si hizo asistencia)
rating       = clamp(RATING_BASE + (factor - 1) × RATING_FACTOR_COEFICIENTE + golesPartido × 0.7 +
0.4 (si asistencia) + ruido(±0.4), 5, 10)
              (RATING_BASE = 6.5, RATING_FACTOR_COEFICIENTE = 1.3)
```

Con el coeficiente viejo (2.5), un debutante de bajo OVR (factor ~0.3-0.5) promediaba notas de **~5.0-5.3 la temporada entera** — pegado al piso de 5, algo que casi no pasa en la vida real: ni un jugador flojo de verdad promedia tan bajo con continuidad, esas notas son para un partido puntual desastroso, no la norma de toda una temporada. Bajado a 1.3, ese mismo debutante ahora promedia **~5.6-5.9** (recién arrancando, pero no un desastre), sin tocar el techo — un factor alto (2.5-3.0) sigue empujando la nota hacia el 9-10 en partidos con gol/asistencia. Con talento neutral: OVR 50 → ~5.6, OVR 65 → ~6.2, OVR 73 (neutral) → ~6.7, OVR 90 → ~8.5, OVR 99 → ~9.6.

El resultado del tramo (goles, asistencias, MVPs, suma de ratings) se acumula a las estadísticas de la temporada.

11. Progresión de OVR

`ajustarOvrTramo(ovrActual, rendimientoAcumulado, edad, factorTalentos, potencialTecho)` ([config.ts:1418](#)) — se llama una vez por tramo, después de simular las estadísticas de ese tramo:

```
deltaBase = (0.7 + rendimientoAcumulado / 6) × factorCrecimientoPorEdad(edad) × factorTalentos
declive   = factorDeclivePorEdad(edad) × (2 - factorTalentos)
deltaCrudo = deltaBase - declive
si deltaCrudo > 0 y ovrActual + deltaCrudo > potencialTecho:
    deltaCrudo -= exceso × (1 - POTENCIAL_TECNO_FACTOR_MIN)  (recorta lo que se pasaría del techo,
    deja pasar un resto)
delta = redondeoEstocastico(deltaCrudo), recortado entre:
    · [-1, +3] en circunstancias normales (sin declive por edad)
    · [-10, +3] si ya hay declive por edad actuando
ovrNuevo = clamp(ovrActual + delta, 45, 99)  (OVR_CARRERA_MIN / MAX - piso y techo absolutos)
```

Redondeo estocástico ([config.ts:1338](#)): en vez de redondear siempre igual, un valor de 0.4 da +1 el 40% de las veces y 0 el 60% restante — así los cambios chicos de vez en cuando pasan, en vez de quedar completamente anulados por el redondeo.

11.1 Curva de edad en 3 etapas

Antes el freno de crecimiento se estabilizaba en 35% del ritmo pleno **para siempre** desde los 32 años, y el desgaste natural era demasiado débil para competirle — un jugador con buen rendimiento seguía subiendo bastante incluso pasados los 35-40. Ahora:

Freno de crecimiento por edad — `factorCrecimientoPorEdad(edad)` ([config.ts:1281](#)):

Etapas	Edad	Factor
Prime	≤ 28 años	1.0 (crecimiento pleno)
Meseta	29 a 34	interpola linealmente de 1.0 a 0.15
Ocaso	35+	fijo en 0.15 (un resto mínimo — nunca llega a cero del todo)

Desgaste natural por edad — `factorDeclivePorEdad(edad)` ([config.ts:1327](#)), superpuesto a la meseta de arriba en vez de arrancar recién cuando termina:

- Antes de los 30 años: 0 (sin desgaste).
- De 30 a 37: resta **0.06 de OVR por tramo, por cada año** por encima de 30.

- De 37 en adelante: además, resta **0.22 por tramo, por cada año** por encima de 37 (la caída se acelera).

Con estos números, el neto (crecimiento – desgaste) pasa de "todavía sumas algo" a "cuesta mantenerte" de forma gradual dentro de la ventana 29-34, en vez de un quiebre brusco a los 32 — el pico típico de una carrera queda entre los 30-33 años, y para el retiro obligatorio (41-45) ya bajó de forma notable. En cuanto el desgaste es mayor a 0, el piso de variación por tramo pasa de -1 a **-10** (`OVR_TRAMO_DECLIVE_VARIACION_MIN`).

11.2 Talento oculto y techo de potencial

Talento oculto: al arrancar la carrera se sortea, una única vez, un multiplicador entre **0.85x y 1.2x** (`TALENTO_MIN / MAX`, `config.ts:1367-1368`; `GameConfig.sortearFactorTalento()`, sorteado en `career.ts:206`) que acelera el crecimiento (`deltaBase`) y, invertido, atenúa el desgaste ($\times (2 - \text{factorTalento})$): 1.2 lo deja en 80%, 0.85 lo agrava a 115% — con las mismas decisiones de punta a punta, dos carreras no crecen (ni declinan) exactamente igual.

El mismo `factorTalento` también pesa en `simularTramo` (ver [sección 10](#)) — antes solo tocaba el crecimiento de OVR, así que a igual OVR, TODA carrera rendía exactamente igual en cancha, y la única narrativa posible era "malo que se hizo bueno". Ahora un talento alto ya rinde mejor con el mismo OVR bajo (se nota que tiene algo especial antes de que el número lo confirme), mientras uno bajo sigue leyéndose como el grinder clásico. Para que la comparación de los premios mundiales ([sección 14.2](#)) siga siendo pareja, cada candidato del pool sortea su propio `factorTalento` independiente — si no, el tuyo sería una ventaja permanente contra un pool que nunca lo tiene.

Techo de potencial (`potencialTecho`, sorteado una única vez en `career.ts:207`, nunca expuesto en ningún número visible): sin esto, el crecimiento del prime empujaba casi cualquier carrera por encima de 90 — no era una excepción, era casi aritmética garantizada. `sortearPotencialTecho()` (`config.ts:1407`) reparte:

Probabilidad	Techo sorteado	Lectura
30%	70-79	Jugador modesto, nunca despegas del todo
50%	80-85	Profesional sólido
20%	86-98	Estrella de élite

El crecimiento no se frena "acercándose" al techo (esa fue la primera versión probada — combinada con el freno de edad de la misma ventana 29-34, casi nadie llegaba cerca de un techo alto a tiempo). En cambio, actúa a pleno ritmo hasta el final, y `ajustarOvrTramo` solo recorta lo que un tramo puntual se pasaría de largo del techo, dejando pasar un resto (`POTENCIAL_TECNO_FACTOR_MIN = 0.08`, un 8%) — así una racha buenísima puede "sorprender" y pasarlo por uno o dos puntos en casos raros. Los rangos de la tabla de arriba no son directamente "dónde termina la carrera" (varias con techo alto se quedan cortas por el camino, sea por mala racha o por no alcanzar el límite superior del rango) — se

calibraron corriendo **600 carreras completas simuladas de punta a punta** (vía el store real, con un jugador que no se retira apenas puede sino que sigue fichando mientras haya oferta) hasta que el **pico final** de OVR quedara repartido de forma realista. Antes (5%/55%/40%, alto 91-98) el pico medido terminaba **~4% por debajo de 80, ~19% en 80-85, ~77% en 86+ (35% ya por encima de 90)** — mucho más corrido hacia arriba de lo que se sentía como intención de diseño. Con los valores actuales, el pico queda **~25% por debajo de 80, ~53% en 80-85, ~22% en 86+ (~10% por encima de 90)** — la mayoría de las carreras, incluso jugando bien, terminan como jugadores sólidos de 80s, no como crack de 90+.

Al fichar por un club nuevo se garantiza un mínimo margen de crecimiento sobre el OVR inicial (`potencialTecho = max(sorteo, ovrInicial + 5)`) — rarísimo que choquen, pero un debutante en un club grande puede arrancar con 65, y sin este piso una tirada floja del techo lo dejaría prácticamente congelado desde el primer tramo.

11.3 Salto de calidad del arranque de carrera

Sin esto, un debutante de club humilde (OVR ~50-55) tardaba **8-12 temporadas** —de una carrera de ~26-27— en cruzar el umbral de OVR 70, más de un tercio del juego entero rindiendo con números flojos por partida doble (poco OVR y, encima, poco `factorPorFuerzaLiga` de la sección anterior). `factorAprendizajeJoven(ovr, edad)` (`config.ts:1307`) acelera específicamente esa salida, sin tocar ninguna de las 3 etapas de `factorCrecimientoPorEdad` ni el declive por edad:

```
progreso = clamp((UMBRAL_CRECIMIENTO_ACCELERADO - ovr) / (UMBRAL_CRECIMIENTO_ACCELERADO - OVR_CARRERA_MIN), 0, 1)
factorAprendizaje = 1 + progreso × (CRECIMIENTO_ACCELERADO_FACTOR_MAX - 1)
(UMBRAL = 72, FACTOR_MAX = 1.8 — solo si edad ≤ OVR_EDAD_PRIME_MAX)
```

Dos guardas para no romper el resto del sistema de crecimiento (dentro de

`factorAprendizajeJoven`, `config.ts:1307`):

- **Solo aplica en Prime** (edad ≤ 28, donde el crecimiento ya es pleno) — un veterano que bajó de nivel en la meseta o el ocaso nunca lo activa, aunque su OVR haya caído por debajo del umbral. El "salto de calidad" es cosa de un jugador que recién arranca, no de alguien en decadencia.
- **Solo multiplica el crecimiento cuando ya es positivo** — nunca agrava una racha floja de decisiones; una mala temporada declina exactamente igual que antes.

Con esto, las mismas 8-12 temporadas para cruzar OVR 70 bajan a **~5-9**, según qué tan floja sea la liga/club inicial — la etapa de jugador limitado se acorta, no desaparece.

12. Estado de forma

7 estados (`FORM_STATES` , `config.ts:684`), de mejor a peor: **Inspirado** 🔥 → **En plenitud** 💪 → **Animado** 😊 → **Regular** 😐 → **Desanimado** 😞 → **Bajo de forma** 📉 → **Tocado físicamente** 🤕.

Cada opción de cada decisión de evento define un **objetivo** de forma fijo (`efectos.forma` , no es aleatorio — está definido evento por evento en `data/events.ts`), pero ya no la teletransporta ahí directamente: `acumularForma(formaActual, formaObjetivo)` (`config.ts:707`) te acerca a ese objetivo recorriendo una fracción del camino (`FORMA_PESO_ACUMULACION = 0.5` del tramo que falta, con un paso mínimo de 1 escalón para no estancarse justo antes de llegar). Así, dos decisiones seguidas que tiran para el mismo lado siguen sumando en vez de que la segunda pise a la primera, y si tiran para lados opuestos se combinan en vez de que gane la última resuelta. La forma afecta:

- **Participación** (tabla en [sección 9](#)).
- **La "calidad" de la temporada del equipo** (`FORMA_CALIDAD` , ver [sección 14](#)) — de 1.0 (inspirado) a 0.05 (lesionado).

Mientras hay una lesión de nivel 1 o 2 activa, la forma queda **fija en "Tocado físicamente"** sin importar qué decisiones tomes (las decisiones siguen sumando a `rendimiento / equipo` , solo no "curan" el ánimo de golpe) — `career.ts:1105`.

13. Lesiones

Se evalúan en **cada pausa de decisión** (nunca en una de fichajes), solo si no hay ya una lesión activa — `intentarGenerarLesion(edad)` (`career.ts:341`).

Probabilidad de lesión — `probabilidadLesion(edad)` (`config.ts:1969`):

```
prob = clamp(0.065 + max(0, edad - 30) × 0.0027, 0, 0.18)
```

Sube con la edad a partir de los 30 años, con un techo del 18%.

Nivel de la lesión (sorteo ponderado, `elegirNivelLesion`, `config.ts:1979`):

Nivel	Probabilidad de que salga	Efecto
Nivel 3 (leve)	55%	Sin partidos durante 1 pausa. Sin efecto en forma ni OVR.
Nivel 2 (moderada)	35%	Sin partidos 1–2 pausas + forma fija en "Tocado" + OVR -1 a -3 , aplicado de una vez.
Nivel 1 (grave)	10%	Sin partidos, entre 2 pausas y el resto de la temporada + forma fija en "Tocado" + OVR -4 a -10 , aplicado de una vez.

La duración exacta (`duracionLesion`, `config.ts:2002`) y la pérdida de OVR (`ovrPerdidoPorLesion`, `config.ts:2015`) se sortean dentro de esos rangos, siempre topeados por los tramos que en verdad quedan en la temporada. Cada nivel tiene su propio banco de nombres/descripciones de lesión real (rotura de LCA, esguince, desgarró, etc. — ver [sección 20](#)).

Cuando sale una lesión nueva, esa pausa entera se reemplaza por un **parte médico** (sin decisiones que tomar, `LesionCardItem.vue`, con el tipo `InformeLesion` marcado `esInformeLesion: true` en `career-types.ts:72-73`): muestra nombre, descripción, OVR perdido (si corresponde) y pausas de baja, y el jugador confirma con un botón **"Continuar"** para avanzar el tramo — no hay temporizador ni avance automático. En mobile esa tarjeta ocupa el ancho completo del carrusel de decisiones en vez de compartir espacio como una tarjeta más.

Recuperación al darte de alta: al cumplirse los tramos de baja, se te devuelve el **50%** del OVR que perdiste por esa lesión (`LESION_RECUPERACION_OVR`, `config.ts:2024`, aplicado en `career.ts:688`) — fue un golpe físico puntual, no una pérdida de nivel definitiva. El toast de "te recuperaste" muestra cuánto OVR recuperaste, si fue mayor a 0.

14. Sistema de competencias (liga, copas, clasificación internacional)

14.0 Antes de esto: la clasificación de clubes y ligas

Cada club y cada liga en `GameDatabase` tiene **3 ejes ocultos de 0 a 100** (reemplazan al viejo `nivel` entero 1-3/1-6):

Eje	Qué mide	Dónde se usa
<code>fuerza</code>	Nivel deportivo actual del plantel/competencia	Quién gana títulos (sección 14, acá mismo)
<code>prestigio</code>	Historia, títulos, marca, hinchada	Valor de mercado (sección 15) y qué tan aspiracional es un destino (sección 16)
<code>economia</code>	Poder financiero (presupuesto, sueldos, TV)	Valor de mercado y ofertas

Se combinan de dos formas distintas, a propósito, según qué le corresponde a cada mecánica (`config.ts:440-677`):

- `poderEquipo / poderLiga` (`PODER_PESO_FUERZA=0.4` / `PODER_PESO_PRESTIGIO=0.35` / `PODER_PESO_ECONOMIA=0.25` , `config.ts:440/:454-464`): mezcla los 3 ejes para todo lo que en la vida real depende de una combinación de las tres cosas a la vez — el OVR inicial, la ventana de ofertas, el objetivo de prestigio del jugador.
- `calidadFuerzaClub` (`config.ts:1567`, sección 14.1 acá abajo): usa **solo fuerza** — ganar títulos depende de qué tan fuerte es el plantel hoy, no de cuánta plata tiene el club ni de su prestigio histórico.
- `valorScoreEquipo / valorScoreLiga` (`config.ts:632-639`, sección 15): usan **solo economia + prestigio** — cuánto vales en el mercado depende de la plata y la marca del club que te tiene, no de si ese club está ganando esta temporada.

La economía de un club nunca cae debajo del 60% de la de su propia liga (`ECONOMIA_PISO_LIGA` , `economiaEfectiva` , `config.ts:448-452`) — un club chico de una liga rica igual tiene más plata que casi cualquier gigante de una liga menor, por el reparto de TV.

Los ~25 clubes más reconocibles del mundo (Real Madrid, Boca Juniors, PSG, Newcastle, etc.) tienen estos 3 valores puestos a mano; el resto se generó una sola vez a partir de su antiguo nivel 1-3 y la liga a la que pertenece, con una variación estable por club (mismo id → siempre el mismo resultado) para

que no todos los equipos de un mismo nivel queden con el número idéntico — ver el comentario de formato en [data/database.ts:9-38](#).

14.1 Fuerza de campaña y fuerza de título

Ya no es un único número por temporada — hay **dos**, calculados con los mismos 4 ingredientes pero pesados distinto según para qué se usan.

Fuerza de campaña — `calcularFuerzaCampana(equipo, liga, forma, equipoAcumuladoTemporada, promedioJugador)` ([config.ts:1577](#)):

```
calidadClub          = calidadFuerzaClub(equipo, liga)  (SOLO el eje fuerza, 55% equipo + 45%
liga)
calidadForma         = FORMA_CALIDAD[forma]          (1.0 inspirado ... 0.05 lesionado)
calidadEquipoAcum    = clamp(0.5 + equipoAcumuladoTemporada / 8, 0, 1)
calidadRendimientoJugador = promedioJugador ? clamp((promedioJugador - 6.0) / 3.0, 0, 1) : 0.5

fuerza = 0.4 × calidadClub + 0.15 × calidadForma + 0.2 × calidadEquipoAcum + 0.25 ×
calidadRendimientoJugador
```

`equipoAcumuladoTemporada` es la suma de todos los efectos `equipo` de las decisiones tomadas en la temporada. `promedioJugador` es `temporadaActual.promedio` (el rating medio por partido, que ya arrastra goles/asistencias/MVP — [sección 10](#)) — `null` si todavía no jugaste ningún partido esta temporada, para no castigar como si hubieras rendido pésimo antes de debutar.

Esta fórmula le da a tu temporada personal (forma + decisiones + rendimiento) el 60% del peso — a propósito, porque **avanzar de ronda en una copa, ganar la copa nacional y clasificar a competición internacional** están pensados para que tu aporte individual pese mucho.

Fuerza de título — `calcularFuerzaTitulo(equipo, liga, forma, equipoAcumuladoTemporada, promedioJugador)` ([config.ts:1613](#)): misma fórmula, pero con `FUERZA_TITULO_PESO_CLUB=0.85` / `_FORMA=0.04` / `_EQUIPO_ACUMULADO=0.05` / `_RENDIMIENTO_JUGADOR=0.06` ([config.ts:1608-1611](#)) — el club pasa a pesar el 85%. Se usa **solo** para **ganar la liga** y para la **final de una copa internacional** ([career.ts:878-910](#)): con el 60/40 de la fuerza de campaña, un club como Bayern München (`calidadClub` ~0.90) con un jugador de rendimiento neutro terminaba con una fuerza de ~0.66, casi igual a la de un club mediano de la misma liga — el equipo más ganador de Europa no se sentía distinto de uno de mitad de tabla. Con el 85% de peso al club, los grandes de verdad son candidatos de entrada al título, y una gran temporada tuya los empuja más arriba todavía (a igualdad de club, una temporada floja da ~12% de ganar la liga contra ~40% de una legendaria — más de 3x de diferencia solo por el rendimiento individual).

Ventaja de dominancia doméstica (`VENTAJA_DOMINANANCIA_DOMESTICA`, [config.ts:1666](#)): incluso con la fuerza de título de arriba, la brecha real de `fuerza` entre algunos clubes y su rival doméstico más cercano es demasiado chica para reflejar su dominio real — Bayern München (93) vs Bayer Leverkusen (88) da ~49% contra ~44% de ganar la Bundesliga en un escenario neutro, casi una moneda al aire. Ni poniéndole 100 de fuerza a Bayern alcanza un techo realista (el 15% de peso de

`calcularFuerzaTitulo` que no depende de `calidadClub` lo capta en ~55%), así que **Bayern Múnchen** y **PSG** tienen un bonus fijo de `+0.08` sumado a `fuerzaTitulo`, aplicado **solo** al tirar el título de **liga** (`career.ts:881-883`, nunca a la copa nacional ni a la final de copa internacional de arriba). Con el bonus, Bayern pasa a ~66% y PSG a ~58% (levemente menos — su monopolio real tuvo más sobresaltos, como Mónaco 2017 o Lille 2021) sin tocar las chances de sus rivales domésticos.

De ahí salen 4 cosas:

Ganar la liga (al cierre de temporada, sobre la fuerza de **título**) — curva empinada, casi exclusiva de los grandes — `probGanarLiga` (`config.ts:1635`):

```
prob = clamp(0.02 + 0.85 × fuerzaTitulo^3.5, 0, 0.85)
```

Ganar la copa nacional (si llegaste a la final, sobre la fuerza de **campeña**) — mucho más pareja a propósito — `probGanarCopa` (`config.ts:1641`):

```
prob = clamp(0.05 + 0.70 × fuerza^1.3, 0, 0.70)
```

Ganar la final de una copa internacional (sobre la fuerza de **título**) — misma curva que la liga, `probGanarLiga(fuerzaTitulo)`.

Avanzar de ronda en una eliminatoria (copa nacional o internacional, sobre la fuerza de **campeña**), una tirada por ronda — `probAvanzarRonda` (`config.ts:1648`):

```
prob = clamp(0.25 + 0.5 × fuerza, 0.1, 0.85)
```

Clasificación internacional para la próxima temporada (sobre la fuerza de **campeña**):

- `fuerza ≥ 0.72` o ganaste la liga → clasificas a la competición de **primer nivel** de tu confederación (Champions League / Libertadores / Concacaf Champions Cup / AFC Champions League Elite).
- `fuerza ≥ 0.45` o ganaste la copa nacional → clasificas a la de **segundo nivel** (Europa League / Sudamericana / AFC Champions League Two — CONCACAF todavía no tiene equivalente).
- Si no, no clasificas a nada.

Las confederaciones con liga(s) cargada(s) son `UEFA` / `CONMEBOL` / `CONCACAF` / `AFC` (`CAF` todavía no tiene ninguna liga doméstica propia, solo selecciones — ver [sección 19](#)). Si tu confederación no tiene competición de un nivel dado (el caso de CONCACAF sin segundo nivel), simplemente no clasificas a nada en ese nivel — no hay error ni sustituto.

Los partidos de cada competición (mínimos garantizados + extra por ronda) salen de `GameDatabase.competiciones` — ver [sección 21](#).

14.2 Premios mundiales: Bota de Oro, Once Ideal y Balón de Oro

Este juego no simula miles de jugadores rivales por el mundo — solo existe tu propio personaje. Para que "eres el mejor del mundo" signifique algo real, al cierre de cada temporada

(`generarCandidatosPremiosMundiales` , `career.ts:775`, llamado desde `finalizarTemporada` después de resolver liga/copa/copa internacional) se genera un pool de **24 candidatos fantasma** de nivel élite, simulados con **la misma fórmula que usa tu propio jugador**

(`GameConfig.simularTramo`) — así la comparación es justa: si tus números se sienten inflados o flojos, los del pool se sienten exactamente igual.

Generación de cada candidato (`PREMIOS_CANDIDATOS_N = 24` , `config.ts:1687`):

- **OVR:** `sortearOvrCandidatoPremio()` (`config.ts:1708`) — entre 82 y 99 (`PREMIOS_OVR_MIN/MAX`), promediando 3 tiradas uniformes para sesgar hacia el centro del rango (85-95) en vez de una muestra pareja — son candidatos genuinos al premio, no cualquier nivel élite. Se sortea **antes** que el club, porque el club depende de él (ver el punto siguiente).
- **Club (liga + equipo):** elegido con la misma "ventana de OVR"/cercanía de nivel que ya usa el sistema de fichajes real (`equipoElegibleParaOvr` / `pesoPorCercaniaNivel` / `elegirMejorEncaje` , sección 16.5/16.6), sobre todos los equipos de las ligas con competición doméstica cargada — así un candidato de 96 OVR aparece casi siempre en uno de los pocos clubes de ese nivel, y sirve además para narrar el mensaje con sentido ("un delantero de Bayern Múnich..."). Antes el club se elegía primero (liga ponderada por `liga.fuerza` , equipo dentro de ella por `equipo.fuerza`) totalmente al margen del OVR sorteado después — podía salir, por ejemplo, un candidato de 96 OVR y 38 goles en un club chico sin poder real para eso.
- **Grupo de posición:** `sortearGrupoCandidatoPremio()` (`config.ts:1713`) — 55% ataque / 25% medio / 12% lateral / 8% central (`PREMIOS_PESO_GRUPO`); nunca arquero, nadie gana la Bota de Oro de arquero.
- **Talento oculto:** `sortearFactorTalento()` — cada candidato sortea el suyo propio, igual rango que el tuyo (0.85x-1.2x), para que la comparación siga siendo pareja (ver sección 11.2).
- **Estadísticas:** una sola llamada a `simularTramo({ partidos: partidosMinimos de su liga, posicion: POSICION_REPRESENTATIVA_GRUPO[grupo], ovr, rendimientoAcumulado: 0, fuerzaLiga, factorTalento })` (`career.ts:787-794`, rendimiento neutro — no tiene decisiones propias que tomar). `simularTramo` ya no toma un grupo de 5 perfiles sino una posición individual (ver sección 10), así que cada candidato usa la posición más "típica" de su grupo sorteado — `POSICION_REPRESENTATIVA_GRUPO` (`config.ts:1703`): DC para ataque, MCO para medio, LI para lateral, DFC para central, POR para arquero.
- **Trofeos:** `Math.random() < probGanarLiga(calidadFuerzaClub(equipo, liga))` (`career.ts:795`) — reutiliza la misma curva que decide si TU club gana la liga, en vez de inventar una probabilidad aparte.

 **Bota de Oro:** tu `goles` de la temporada contra el máximo del pool, sin filtrar por posición (un jugador de otro grupo con pocos goles nunca compite en la práctica, sin necesidad de un caso especial). Si no ganas pero quedas entre los 3 mejores, un mensaje aparte ("Terminaste 2° en la Bota de Oro, detrás de un delantero de PSG con 34 goles").

★ **Once Ideal:** ganar la Bota de Oro o el Balón de Oro (ver abajo) ya te garantiza un lugar acá — no tendría sentido ser el goleador o el mejor jugador del mundo y quedar afuera del mejor 11 de tu propia posición. Si no ganaste ninguno de los dos, se evalúa como siempre: tu `promedio` de rating contra los candidatos de **tu mismo grupo de posición** — no simula quién ocupa los otros 10 puestos, igual que el juego no simula las otras 31 selecciones en un Mundial. Con un margen de tolerancia (`ONCE_IDEAL_MARGEN_PROMEDIO = 0.4`): desde OVR ~95 el rating de cada partido queda clampeado al tope (10.0) sin variación posible (ver [sección 10](#)), así que comparar el promedio exacto dejaba el premio reservado casi solo a quien pisa ese umbral literal — con el margen, un promedio de élite real un poco por debajo (9.5-9.9) también tiene una chance genuina, no solo cero o cien por ciento.

🏆 **Balón de Oro:** un puntaje combinado contra TODO el pool, sin importar posición — `calcularCalidadBalonDeOro(promedio, goles + asistencias, ganoTrofeo)` ([config.ts:1739](#)):

```
calidadRating    = clamp((promedio - 7.0) / 2.5, 0, 1)
calidadGoleador  = clamp((goles + asistencias) / 40, 0, 1)    (BALON_ORO_REFERENCIA_GOLES)
calidadTrofeos   = ganaste algo esta temporada ? 1 : 0
calidad = 0.4 × calidadRating + 0.35 × calidadGoleador + 0.25 × calidadTrofeos
```

Ninguna pata sola alcanza — hace falta rendimiento de élite **y** producción goleadora **y** haber ganado algo, las tres a la vez. Es el más difícil de los tres.

`ganoTrofeo` (para el jugador) es `ganasteTrofeoDeEquipo0Seleccion`, capturado en `finalizarTemporada` **antes** de otorgar la Bota de Oro o el Once Ideal ([career.ts:912](#), pasado como parámetro a `evaluarPremiosMundiales`) — no se lee en vivo desde `temporadaActual.trofeos.length > 0` dentro de la misma función que evalúa los tres premios. La razón: Bota de Oro y Once Ideal NO requieren haber ganado nada de equipo, y esta misma función los agrega a ese array un poco más abajo — si el Balón de Oro mirara el array en ese momento, ganar cualquiera de esos dos premios individuales "contaría como trofeo" para el propio Balón de Oro, volviendo casi automático un barrido de los tres sin haber ganado una sola liga, copa o título con la selección (medido: 70% de las veces con estadísticas de élite y cero trofeos reales, contra 0.15% ya corregido).

Los tres empates (`≥` en vez de `>` en las tres comparaciones) los gana el jugador — dado el clampeo de rating de arriba, un empate exacto contra el pool es común en el tramo alto, y no tendría sentido que ese empate SIEMPRE lo pierda el jugador. Los trofeos ganados se guardan en el mismo array `temporadaActual.trofeos` que los de liga/copa/selección — reutilizan toda la UI existente sin cambios (badge, historial, resumen, tarjeta para compartir). Los tres premios ya tienen ícono propio en `assets/escudos/trofeos/`: `bota-de-oro.png`, `balon-de-oro.png` y `once-ideal.png`.

14.3 Rival de carrera (motor activo, oculto en la interfaz)

Los 24 candidatos a premios mundiales de arriba se generan y se descartan cada temporada — nunca quedan registrados en ningún lado. El rival reusa esa misma lógica de generación, pero **una sola vez**, al arrancar la carrera, y persiste: es un candidato de nivel élite que "empezó su carrera el mismo día que tú" y crece en paralelo, temporada a temporada, sin decisiones propias que tomar.

Oculto a pedido, no eliminado: el feature completo (generación, avance temporada a temporada, persistencia) sigue corriendo tal cual en cada carrera, pero **ninguna pantalla lo muestra hoy** — se sacaron tanto el mensaje comparativo al cerrar cada temporada como la sección "Tu rival" del resumen final, sin tocar el motor por si se reactiva más adelante. Si volviera a mostrarse, no haría falta ningún cambio de datos: `rival` ya se genera, avanza y persiste en cada carrera existente.

Generación (`generarRival(edadJugador)` , `career.ts:753-773`, llamado una sola vez desde `iniciarCarrera` , `career.ts:220`): mismo pool y mismo criterio de nivel que un candidato a premio (`sortearOvrCandidatoPremio` , `equipoElegibleParaOvr` + `pesoPorCercaniaNivel` + `elegirMejorEncaje` , `sortearGrupoCandidatoPremio`) — de hecho, la parte de "elegir club + grupo de posición para un candidato élite" está factorizada en un helper compartido, `elegirClubYGrupoElite(pool, ovr)` (`career.ts:736-748`), usado tanto acá como en `generarCandidatosPremiosMundiales` (arriba), sobre el mismo pool de equipos (`poolEquiposElite()` , `career.ts:718-728`). Sin nombre propio inventado — se lo referencia siempre como "un/a [posición] de [Club]" (mismo tono que ya usa el mensaje de subcampeón de Bota de Oro). A diferencia de un candidato a premio, el rival sí sortea su propio `factorTalento` y `potencialTecho` (`sortearFactorTalento` / `sortearPotencialTecho` , las mismas funciones que usa el propio jugador), porque los va a necesitar para crecer temporada a temporada.

Avance, en `finalizarTemporada` (`career.ts:926-946`, justo después de resolver los premios mundiales de la temporada): una sola llamada a `simularTramo` (mismo patrón que un candidato a premio, `rendimientoAcumulado: 0` porque no tiene decisiones propias) le suma goles/asistencias de esa temporada a sus totales de carrera, y `ajustarOvrTramo(ovr, 0, edadRival, factorTalento, potencialTecho)` — la misma fórmula de crecimiento de OVR que usa el propio jugador, reusada tal cual — le sube el OVR. **No** se simulan trofeos, lesiones ni convocatorias a selección del rival — a propósito, para mantener el feature chico: solo importan OVR, goles y asistencias de carrera.

Estado: `Rival` (`career-types.ts:187-198`) es un objeto en el store (`rival` , `null` si no hay ligas con competición doméstica cargada), persistido en `guardar()` / `cargar()` igual que el resto del estado — ver [sección 23](#).

15. Valor de mercado

`calcularValorMercado(ovr, equipo, liga)` (`config.ts:649`) — curva exponencial sobre el OVR (cada punto extra cerca del techo vale desproporcionadamente más, como en la vida real), multiplicada por la economía y el prestigio del club/liga actual (a propósito, **no** por su fuerza — ver [sección 14.0](#)):

```
valorPorOvr = 3,000,000 × 1.08^(ovr - 45)      (VALOR_MERCADO_BASE, VALOR_MERCADO_CRECIMIENTO, config.ts:623-624)
valorScore = 0.6 × economía-efectiva + 0.4 × prestigio      (VALOR_PESO_ECONOMIA / _PRESTIGIO, valorScoreEquipo/valorScoreLiga, config.ts:632-639)
multiplicadorClub = interpola entre 0.5 (valorScore más flojo) y 1.4 (más alto)
                    combinando equipo y liga (55% / 45%, calcularMultiplicadorClub, config.ts:641-647)
valor = round(valorPorOvr × multiplicadorClub / 1000) × 1000      (redondeado al millar)
```

`VALOR_MERCADO_BASE` / `_CRECIMIENTO` arrancaron en 18000/1.185, pero esa curva era demasiado "back-loaded": de OVR 90 a 99 el valor se multiplicaba por 4.6x, así que un 90 real — ya un techo raro, la mayoría de las carreras buenas terminan entre 80 y 95 (ver [sección 11.2](#)) — quedaba infravalorado en plata (~€50M jugando en el Real Madrid, muy por debajo de lo que paga el mercado real por ese nivel). Con 3,000,000/1.08 la franja "estrella real" (85-92) sube mucho más en términos relativos, mientras el techo casi mítico (OVR 99) se mantiene en un rango creíble en vez de dispararse todavía más:

OVR	Real Madrid (multiplicadorClub ≈ 1.36)
45 (piso)	~€4.1M
60	~€12.9M
80	~€60.2M
90	~€130M
95	~€191M
99 (techo casi mítico)	~€260M

Se recalcula cada vez que cambia el OVR (cada tramo) y cada vez que cambias de club (con la economía/prestigio del club nuevo). Se muestra formateado con `formatMarketValue` (`game/format.ts:38`): `€4.1M`, `€130M`, etc.

Para las **ofertas de fichaje** se usa una variante cosmética, `valorOfrecidoPorClub` (`config.ts:661`), que le suma un ±8% de variación aleatoria (`OFERTA_VARIACION_VALOR`) — para que dos clubes de poder parecido no muestren el mismísimo número al centavo en sus tarjetas. No afecta tu valor de mercado real, solo el texto "Te valoran en €X" de esa tarjeta puntual.

Reputación (fama) — un eje DISTINTO del OVR, a propósito: puedes ser muy bueno y poco conocido, o al revés. Escala acotada 0-100 igual que el resto de los medidores del juego, monotónica (solo sube, nunca decae — no hay "mala prensa" que reste en esta versión). Dos fuentes:

- **Decisiones de prensa:** `EfectosOpcion` ganó un cuarto campo opcional, `fama?: number` (`events.ts:71`) — queda en 0 sin tocar para las 200 entradas que no lo usan, y está curado a mano (valores chicos, de -2 a +3, algún -4 en los eventos de alto impacto) en las ~26 entradas con `personajes: ["prensa"]`. Se aplica en `resolveDecisionEvento` (`career.ts:1110-1115`), clampeado a `[0, FAMA_MAX]`.
- **Trofeos de la temporada:** al cerrar cada temporada, cada trofeo en `t.trofeos` (que en ese punto ya incluye los de club/selección y los premios individuales que acaba de agregar `evaluarPremiosMundiales`, sección 14.2) suma `FAMA_POR_TROFEO` o `FAMA_POR_PREMIO_INDIVIDUAL` si es Bota de Oro/Balón de Oro/Once Ideal (`career.ts:916-924`):

```
FAMA_POR_TROFEO: 2          (config.ts:2046)
FAMA_POR_PREMIO_INDIVIDUAL: 6 (config.ts:2047, pesa más que un trofeo de equipo)
FAMA_MAX: 100              (config.ts:2045)
```

Se muestra como una **etiqueta de nivel**, no como número crudo — a propósito, para que se lea como un eje distinto del OVR y no compita con él como "el número importante".

`famaTierLabel` / `famaTierColor` (`game/format.ts:20-37`) mapean 6 niveles (Anónimo, Regional, Conocido, Estrella, Ídolo, Leyenda mundial) con una paleta de color propia — deliberadamente distinta de `ovrTierColor` (bronce/plata/oro/zafiro/rubí/amatista), para no leerse como el mismo medidor. Aparece como chip en el hero (`HeroPanel.vue:80-82`) y como tile en el resumen final de carrera (`ResumenModal.vue:157`) — ver sección 22.

16. Sistema de fichajes y ofertas

Toda la lógica vive en `generarLoteOfertas()` ([career.ts:376](#)), que corre en la única pausa de "oferta" de cada temporada (ver [sección 7](#)).

16.1 Retiro forzoso (el corte final)

```
if (edad ≥ edadRetiroForzoso) return [ solo la carta de retiro forzoso ];
```

`edadRetiroForzoso` se sortea **una sola vez por carrera**, entre 41 y 45 años (`EDAD_RETIRO_FORZOSO_MIN / MAX`). A partir de esa edad, no importa el club ni el OVR: la única carta es retirarte.

Transición previa (no es un corte seco): en las **2 temporadas** justo antes de esa edad (`EDAD_RETIRO_TRANSICION`, [config.ts:944](#)), el cupo de ofertas de club se reduce a **1** en vez de 2 — cada vez menos clubes se animan a día ofertarte, hasta que en la última temporada esa única oferta también desaparece. Se implementa como una tercera categoría de cupo en [career.ts:454-455](#): `enTransicionRetiro` reduce `cantidadOfertasClub` a 1 (solo si el contrato actual sigue en pie), antes de llegar al corte total de la edad forzosa.

16.2 Período de gracia de contrato

```
graciaContrato = esPrimerClub ? 4 : 2 (TEMPORADAS_GRACIA_CONTRATO_PRIMER_CLUB /  
TEMPORADAS_GRACIA_CONTRATO)  
enGraciaDeContrato = temporadasEnClubActual < graciaContrato
```

Mientras estés en gracia, tu club **nunca** puede "no renovarte" — sin este colchón, cualquier club de nivel medio/alto para arriba te dejaría ir en tu primerísima ventana de fichajes, porque ningún novato arranca con el OVR de un jugador hecho (ver [sección 5](#): tope de 65 vs. ventanas de OVR que fácilmente piden 70+). El contador (`temporadasEnClubActual`) se resetea a 0 cada vez que fichas por otro club y sube +1 en cada cierre de temporada en el mismo club.

El período de gracia es distinto para el **primer club de la carrera** (el de la creación de personaje): **4 temporadas** en vez de las 2 normales de cualquier club fichado después — es tu debut real, no alguien fichado ya con currículum, así que el club que apostó por ti te da el doble de margen. Se trackea con `esPrimerClub` ([career.ts:117](#), arranca en `true` y pasa a `false` para siempre en el primer traspaso, en `resolveOferta`).

16.3 ¿Tu club actual te renueva?

Pasado el período de gracia, `contratoDebeTerminar(equipo, liga, ovr, promedioTemporadaAnterior)` (`config.ts:974`) compara primero tu OVR contra la "ventana de OVR" de tu propio club (ver 16.5 más abajo): si llegas al mínimo, sigues sin más vueltas. Si no llegas, todavía hay una salida: si el **promedio de rating con el que cerraste la temporada anterior** fue realmente bueno (≥ 7.5 , `CONTRATO_RENDIMIENTO_SALVAVIDAS`, muy por encima del neutral de 6.5) el club te renueva igual — así se puede cerrar una temporada brillante en goles/asistencias/rating sin que el club te corte solo porque el OVR (que crece con su propia curva de edad/techo, no 1 a 1 con las estadísticas del año) no llegó a tiempo. Si ninguna de las dos te salva, no te renuevan — la carta de "Quedarme" se reemplaza por una de retiro (no forzoso, con el texto "decide no renovarte para la próxima temporada").

16.4 Retiro voluntario

```
puedeElegirRetiro = !contratoTerminado && edad ≥ 36      (EDAD_RETIRO_OFERTA)
```

Desde los 36 años puedes elegir colgar los botines aunque tu club te siga queriendo — ocupa una de las 3 cartas de club, dejando solo 2 cupos de fichaje ese año (y, en ese caso puntual, sin la garantía de liga/país local del punto 16.6).

16.5 Elegibilidad real: la "ventana de OVR" de cada club

Cada combinación equipo+liga solo puede ofertarte si tu OVR cae dentro de su ventana — `equipoElegibleParaOvr` / `ventanaOvrOferta` (`config.ts:903-914`):

```
centro = 45 + calidadPoderCombinada(equipo, liga) × 54      (mapeado a todo el rango 45-99 de carrera)
ventana = [ clamp(centro - 13, 45, 99) , clamp(centro + 13, 45, 99) ]      (OFERTA_TOLERANCIA_OVR = 13)
```

Es un **corte duro**, no solo "menos probable": un club chico deja de poder ofertarte en cuanto eres demasiado bueno para él, y uno grande no entra en juego hasta que estás a su altura (con la tolerancia de 13 puntos, los clubes top del mundo ya son alcanzables desde ~86 de OVR). Esta ventana de elegibilidad **no cambió** con el ajuste de pesos de 16.6 — lo que cambió es solo cómo se prioriza/ordena a los ya elegibles, no quién entra al pool.

Además, la oferta tiene que tener sentido en plata: `ofertaTieneValorRazonable(valorActual, valorEnClub)` (`config.ts:673-677`) descarta clubes donde fichar implicaría un desplome de más del 60% de tu valor de mercado actual (`OFERTA_UMBRAL_CAIDA_VALOR = 0.4` , es decir el club tiene que ofrecerte como mínimo el 40% de lo que vales hoy), aunque el margen de OVR lo deje pasar.

Si el cruce de ambos filtros deja el pool vacío (dataset chico o caso límite), se relaja primero el filtro de valor, y si todavía no alcanza, se usan todos los candidatos — nunca se deja al jugador sin ofertas.

16.6 Cuáles de los elegibles aparecen (y en qué orden de prioridad)

Dentro del pool ya elegible, no se sortea parejo entre todos:

1. **Potencial ajustado por edad** (no el OVR real) decide a qué poder de club/liga "apunta" el jugador — `potencialAjustadoPorEdad(ovr, edad)` ([config.ts:1015](#)):

```
17 a 24 años: bono que baja linealmente de +8 (a los 17) a 0 (a los 24) -  
EDAD_POTENCIAL_BONUS_MAX/HASTA  
25 a 30 años: sin ajuste (edad ideal)  
30+ años: penalización de -0.7 de OVR efectivo por cada año por encima de 30
```

Esto **no** toca la elegibilidad real del punto 16.5 (esa sigue siendo puro OVR) — solo decide, entre los clubes ya alcanzables, cuáles se priorizan. A igual OVR, un jugador de 27 años apunta más arriba que uno de 38.

2. Con ese potencial se calcula un **único poderObjetivo** ([config.ts:848](#): más OVR, más poder objetivo), y el peso de cada candidato según qué tan cerca está de ese objetivo (`pesoPorCercaniaNivel`, [config.ts:871](#)):

```
deltaEquipo = (poderEquipo - poderObjetivo) / 10    (PESO_ESCALA_DISTANCIA - lleva el delta,  
deltaLiga   = (poderLiga - poderObjetivo) / 10      en puntos de poder 0-100, a una escala  
chica)  
factor(delta) = 0.05 si delta > 0 (el club/liga "sobra" de poder), 1 si no (PESO_FACTOR_SOBRAR)  
distancia = |deltaEquipo| × factor(deltaEquipo) + |deltaLiga| × factor(deltaLiga) × 0.6  
peso = 1 / (1 + distancia)^2.2
```

Asimétrico a propósito: "quedarte corto" de poder (delta negativo — un club/liga peor de lo que tu potencial pide) pesa la distancia completa, pero "sobrar" (delta positivo, un club/liga mejor) casi no penaliza (factor 0.05 en vez de 1). Con una fórmula simétrica, un club top "se pasaba" del objetivo tanto como un club chico se quedaba corto, y ambos pesaban lo mismo — en la práctica, nunca aparecían los grandes de Europa hasta OVRs absurdamente altos. Con el peso asimétrico, un club mejor que tu objetivo deja de competir en desventaja contra uno que directamente te queda grande.

3. **elegirMejorEncaje** ([config.ts:829](#)): apoyada en `gruposTierAlto` ([config.ts:823](#), ver también 16.7), ordena los candidatos por peso descendente y sortea (ponderado, para que siga habiendo variedad) **solo dentro del 40% superior** (`OFERTA_TOP_ENCAJE_FRACCION`) — así, si tu nivel da para los grandes, van a ser los grandes los que en verdad aparezcan, en vez de perderse en un sorteo parejo contra todo el pool elegible.

16.7 Cuántas ofertas de club, y la garantía de "tu entorno" (condicional)

- **3 cupos de club** normalmente (+ la carta de tu club actual = 4 tarjetas en total).
- **2 cupos** si puedes elegir retiro voluntario (16.4) — ahí no hay garantía de entorno, queda 100% libre.

Con 3 cupos, la garantía de "tu entorno" ya **no es incondicional** — solo se activa si ese entorno sigue siendo un destino de tu nivel:

- `gruposTierAlto(elegibles, pesoFn)` (`config.ts:823`) calcula el mismo 40% superior por peso que usa `elegirMejorEncaje` (16.6) sobre **todos** los elegibles, sin filtrar por entorno.
- Si algún club de tu entorno cae dentro de ese tier alto, se garantizan **hasta 2 ofertas** de ahí — exactamente 2 si hay al menos 2 candidatos en esa intersección, menos si no los hay.
- Si **ningún** club de tu entorno llega al tier alto (tu nivel ya superó a tu liga actual, o a tu país de origen si eres veterano), la garantía **desaparece del todo** — salvo la excepción de abajo para veteranos.

"Tu entorno" es:

- Normalmente, **tu liga actual**.
- Desde los **33 años** (`EDAD_OCASO_RETORNO_PAIS`, `config.ts:1038`), pasa a ser **tu país de origen** — para simular volver a cerrar la carrera en casa, aunque la hayas jugado toda en el exterior.

Retorno nostálgico esporádico (solo veteranos, 33+): si tu país de origen ya no entra en el tier alto (tu nivel lo superó de sobra) pero igual hay clubes elegibles ahí, aparece **como máximo 1** oferta de esos clubes con **30% de probabilidad** por ventana (`PROB_OFERTA_NOSTALGICA`, `config.ts:1039`) — ya no es una garantía, es una posibilidad ocasional de que un club de tu país intente el gesto sentimental de traerte de vuelta, sin que se sienta forzado en cada carrera.

El resto de los cupos (y todo, si no hay candidatos de entorno) sale libre del pool elegible completo, mismo criterio de mejor encaje. Si aun así faltan candidatos distintos, se completa repitiendo clubes antes que mostrar menos ofertas de las que corresponden.

16.8 Orden de las cartas, y resolver la pausa

Las cartas ya **no se mezclan en orden aleatorio**: la carta de tu club actual (quedarme, o el retiro forzoso si no te renuevan) va siempre **primera**; si además aparece la opción de retirarte voluntariamente (16.4), esa va **segunda**. El resto de ofertas de club llena los cupos restantes, en cualquier orden — así el jugador siempre encuentra "seguir acá" (y "retirarme", si corresponde) en el mismo lugar de la fila, en vez de tener que buscarlos entre las demás ofertas.

Un solo clic resuelve toda la pausa (`resolveOferta`, `career.ts:1131`):

- **Retiro** → cierra la carrera (`sección 17`).
- **Fichar por un club nuevo** → la ventana única de fichajes (ver `sección 7`) cae siempre en pretemporada, así que el traspaso arranca la temporada entera de cero con el club nuevo (nunca parte un año en dos filas de historial): se actualiza club, liga, valor de mercado, se reinician `competiciones` desde cero (la clasificación internacional no se hereda — es del club, no tuya), se resetea `temporadasEnClubActual` a 0 y la forma vuelve a "regular".
- **Quedarme** → sin cambios.
- **Préstamo** → ver 16.9 más abajo.

Ninguna de las dos dispara un toast — el nuevo hero/spotlight (o, si te quedas, la ausencia de cambios) ya lo comunica solo; un mensaje de "Fichaste por X"/"Decidiste quedarte en X" se sentiría redundante, la única pausa del juego donde SIEMPRE habría un toast aunque no hubiera nada nuevo que contar.

16.9 Préstamos

Si tu club te quiere a largo plazo pero no te está dando minutos, te cede a otro por una temporada en vez de solo "te vende o te quedas" — como una cesión real de fútbol.

Se decide en la misma (única) pausa de fichajes de arriba, y reemplaza la ventana normal cuando las DOS condiciones siguientes se cumplen a la vez, dentro del período de gracia de contrato (16.2):

```
pesoTitular < 0.35 (PRESTAMO_PESO_TITULAR_UMBRAL – no te estás ganando el puesto)
promedioTemporadaAnterior < 6.0 (PRESTAMO_PROMEDIO_UMBRAL – por debajo del neutral, 6.5)
```

(`clubDebePrestar` , `config.ts:996`). Pasado el período de gracia, sigue rigiendo 16.3 sin cambios — un préstamo solo tiene sentido mientras el club todavía te quiere conservar a largo plazo.

Si corresponde, la pausa muestra solo 2 cartas: "Quedarme" (igual que siempre) y "Préstamo", con un club destino elegido con el mismo criterio de encaje de 16.6 (misma ventana de OVR de 16.5, mismo peso por cercanía de nivel). Aceptarlo actualiza club/competiciones/pesoTitular/forma igual que un traspaso real de 16.8 — pero, a diferencia de un traspaso, **no** resetea `temporadasEnClubActual` ni `esPrimerClub` : sigues siendo del club dueño, así que su reloj de contrato sigue corriendo esa temporada también.

Al cerrar la temporada de préstamo, vuelves automáticamente al club dueño — sin pedirte nada, sin heredar `pesoTitular` (te lo tienes que volver a ganar ahí, no en el club prestado) ni clasificación internacional (es del club dueño, y no hay forma de saber cómo le fue mientras no estabas — mismo criterio que ya usa un traspaso real, que tampoco la hereda).

(`generarLoteOfertas` `career.ts:376` / `resolveOferta` `career.ts:1131` / `finalizarTemporada` `career.ts:866`)

17. Fin de carrera: retiro y resumen

Al aceptar una carta de retiro (`finalizarCarrera` , `career.ts:1197`):

- Se archiva la temporada en curso tal como quedó — **solo si de verdad se jugó algo** (`t.partidos > 0`). El retiro siempre se decide en la pausa de fichajes, que cae al arranque de una temporada nueva (`crearCalendarioTemporada` , progreso 0, antes de simular cualquier tramo — ver [sección 7](#)), así que esa temporada recién creada nunca tiene partidos jugados. Antes se guardaba igual en el historial, dejando siempre una fila fantasma con 0 partidos/goles/asistencias como "última temporada" de toda carrera.
- El spotlight desaparece y el panel de decisiones (`DecisionsPanel.vue`) muestra el mensaje de despedida con **2 botones**:
 - **"Ver resumen de mi carrera"** → abre `ResumenModal.vue` con:
 - **Banner** con el degradado de colores del último club (mismo lenguaje visual que el hero de `CarreraView.vue` , vía `--rb-a / --rb-b`): escudo, nombre, posición, temporadas jugadas, edad de retiro, y el **badge de pico de OVR** (`ovr-badge--hero` , coloreado con `ovrTierColor` — ver [sección 22](#)) a un costado.
 - **Gráfico de evolución de OVR**: un SVG de área + línea (`calcularArcoOvr` , `game/ovr-chart.ts`) con el OVR de cada temporada de punta a punta, coloreado con el mismo color de gema/metall que el badge de pico — la caption de al lado indica "De X a Y" (OVR de la Temporada 1 al pico alcanzado).
 - **Estadísticas combinadas** de **toda** la carrera (todas las filas de `temporadasFinalizadas`): partidos, goles, asistencias, MVP, promedio de rating y mayor valor de mercado, cada una con su ícono.
 - **Clubes**, como un recorrido horizontal con flechas entre escudos (en el orden en que los fichaste, sin repetir) en vez de una grilla suelta — con scroll propio si fueron muchos. El nombre debajo de cada escudo va alineado a la izquierda (`.resumen__club` , estilo scoped de `ResumenModal.vue`), igual que el resto del texto del resumen.
 - **Trofeos**, agrupados por tipo (un solo ícono por trofeo distinto, con un contador "xN" si lo ganaste más de una vez).
 - **"Aceptar"** → `router.push('/')` para volver a la pantalla de creación de personaje y arrancar una carrera nueva (`DecisionsPanel.vue:71`).

Toda la lógica de agregación (incluida la serie de OVR para el gráfico) vive en `construirResumenCarrera()` (`career.ts:1215`).

18. Solicitud de cambio de dorsal

Se habilita al cerrar **cada** temporada ([career.ts:1005-1006](#)) y queda disponible hasta que se use (no hace falta pedirlo en el momento). Un solo pedido por vez.

`probabilidadAceptarCambioNumero(ovr, equipoAcumuladoTemporada)` ([config.ts:1944](#)) — evaluado con el OVR y el rendimiento colectivo **con los que cerró** la temporada anterior, no con los de la nueva (que todavía no jugó nada):

```
prob = clamp(0.3 + (ovr - 45) × 0.008 + equipoAcumuladoTemporada × 0.05, 0.05, 0.95)
```

El número pedido en sí no influye en nada — lo que pesa es tu peso dentro del plantel, no si "el 10 está más pedido" que el 23.

19. Selección nacional

Sistema aparte del banco de eventos (aunque su tarjeta se muestra en el mismo lugar): representa las convocatorias, torneos y estadísticas del jugador con la selección de su país, en paralelo a su carrera de club.

19.1 Selecciones nacionales (`GameDatabase.selecciones`)

47 selecciones — una por cada país de `COUNTRIES` (`data/countries.ts:12`) — cada una con `fuerza` / `prestigio` (mismo eje 0-100 que clubes/ligas, ver [sección 14.0](#)) y su `confederacion` (`UEFA` / `CONMEBOL` / `CONCACAF` / `CAF` / `AFC` — más amplio que el de `ligas`, porque acá entran todos los países de la creación de personaje, no solo los que tienen liga propia cargada). Van a mano según pedigrí futbolístico real: de Brasil (fuerza 92) a El Salvador (fuerza 34).

19.2 Convocatoria

Se sortea **una vez por temporada**, igual mecanismo que Alto Impacto (ver [sección 20](#)): cada país tiene un "OVR de referencia" que te da un 50/50 de ser convocado — `probConvocatoria(ovr, fuerzaSeleccion)` (`config.ts:1767`):

```
umbral = 50 + fuerzaSeleccion * 0.35                                     (UMBRAL_OVR_CONVOCATORIA_BASE/_FACTOR)
prob   = clamp(0.5 + (ovr - umbral) * 0.04, 0.03, 0.95)               (PROB_CONVOCATORIA_PENDIENTE_OVR,
min/max)
```

Cuanto más grande la selección, más alto el OVR que hace falta: a Brasil (fuerza 92, umbral $\approx$ 82) hay que llegarle con un OVR de élite; a Bolivia (fuerza 35, umbral $\approx$ 62) un OVR medio ya empareja. La probabilidad sube/baja de forma lineal alrededor de ese umbral — no es un corte seco, hay una franja real de incertidumbre.

Si el sorteo da que sí, se elige al azar en qué pausa de la temporada cae la convocatoria (igual que Alto Impacto). Si esa pausa **ya** está ocupada por un evento de Alto Impacto de tipo "deportivo", la convocatoria simplemente no se muestra esa vez (caso raro: requiere que ambos coincidan de pausa).

19.3 La tarjeta de convocatoria

Reemplaza el slot "deportivo" de esa pausa (mismo mecanismo de reemplazo que Alto Impacto), con un dilema real de 2 opciones — nunca gratis, mismo principio que el resto del banco de eventos (ver [sección 20](#)):

Opción	Efecto en tu club	Efecto en tu selección
Priorizar la convocatoria	rendimiento +1, equipo -1	Juegas normalmente (ver 19.4)
Cuidar tu lugar en el club	forma: desanimado, equipo +1	0 partidos esa ventana

Se identifica con un 🌐 en la esquina superior derecha de la tarjeta (mismo lugar que el ⚠ de Alto Impacto) y la etiqueta "Selección" en vez de "Deportivo".

19.4 Qué se juega — amistosos, eliminatorias o el torneo grande

El calendario de grandes torneos sale directo del número de temporada, sin estado adicional que guardar (`tipoAnoTorneoSeleccion` , `config.ts:1829`):

```
temporada % 4 == 1 → año de Mundial
temporada % 4 == 3 → año de copa continental (Copa América / Eurocopa / Copa Oro / Copa Africana /
Copa Asiática, según tu confederación)
en cualquier otro año → solo amistosos/eliminatorias, sin trofeo en juego
```

Al aceptar priorizar la convocatoria, `resolverParticipacionSeleccion` (`career.ts:1015`) resuelve todo de un saque (no se reparte en tramos como las copas de club):

- **Año sin torneo:** juegas entre 3 y 5 amistosos (`PARTIDOS_AMISTOSO_SELECCION_MIN/MAX`) — antes era un número fijo (2), sin variación de temporada a temporada.
- **Año de torneo:** la campaña de eliminatorias se juega **siempre**, clasifiques o no — entre 6 y 10 partidos (`PARTIDOS_ELIMINATORIAS_MIN/MAX`). Se tira si tu país **clasifica** (`probClasificarTorneoSeleccion(calidad) = clamp(0.15 + 0.8 × calidad, 0.05, 0.97)`), más parejo que ganarlo:
 - Si no clasifica, la temporada de selección termina ahí.
 - Si clasifica, empieza el torneo en sí: fase de grupos garantizada (3 partidos, `PARTIDOS_FASE_DE_GRUPOS_SELECCION`), con una tirada para avanzar (`probAvanzarFaseDeGruposSeleccion(calidad) = clamp(0.35 + 0.6 × calidad, 0.15, 0.95)`) — bastante generoso, como en la vida real).
 - Si avanza, una ronda eliminatoria por vez (octavos → cuartos → semifinal → final en el Mundial, cuartos → semifinal → final en los continentales), cada una con la **misma** `probAvanzarRonda(calidad)` que ya usan las copas de club ([sección 14.1](#)) — cada ronda se juega (y cuenta como partido) se gane o se pierda, así el partido en el que quedás eliminado también suma. Si gana todas, es **campeón** y el trofeo (Copa del Mundo / Copa América / Eurocopa / Copa Oro / Copa Africana de Naciones / Copa Asiática) se suma a `temporadaActual.trofeos` , el mismo array que los trofeos de club. Si pierde justo la final, queda **subcampeón**.

Las eliminatorias NO se suman a `seleccionPartidos` / `seleccionGoles` / `seleccionAsistencias` — solo quedan mencionadas en el texto del mensaje ("Clasificaste a la Copa del Mundo con Argentina

tras 8 partidos de eliminatorias. Quedaste eliminado en cuartos de final...). Antes se sumaban al mismo total: una carrera podía leer "eliminado en cuartos de final" junto a "14 PJ" con la selección, un número que no tenía relación evidente con "cuartos de final" (un Mundial real, eliminado en cuartos, son 5 partidos — 3 de grupos + octavos + cuartos — las eliminatorias son una campaña aparte, de casi 2 años en la vida real, que nadie cuenta junto con "cuántos partidos jugué en el Mundial"). Ahora `seleccionPartidos` de una temporada de torneo es siempre la cuenta del torneo nomás: 0 si no clasificaste, 3 si quedaste en fase de grupos, y de ahí +1 por cada ronda de eliminación jugada (ganada o perdida) hasta que quedás afuera o sos campeón.

`calidad` es `calidadSeleccion(fuerzaSeleccion, forma)` (`config.ts:1782`): la fuerza fija del país pesa la enorme mayoría, con un empujón chico (`SELECCION_PESO_JUGADOR = 0.15`) según tu forma del momento — un solo jugador no decide el destino de todo un seleccionado.

Los goles y asistencias de esos partidos reutilizan `GameConfig.simularTramo` tal cual la usa el club (misma propensión por posición individual, `sección 10`), para que rendir con la selección se sienta igual que con el club — `career.ts:1080-1087`, solo sobre los partidos del torneo (las eliminatorias no generan goles/asistencias registrados, por la misma razón de arriba).

19.5 Estadísticas y dónde se ven

Cada temporada guarda su propio `seleccionPartidos` / `seleccionGoles` / `seleccionAsistencias` (independientes de los `partidos` / `goles` / `asistencias` de club — nunca se suman entre sí):

- **Temporada en curso:** si hubo convocatoria, el spotlight muestra una línea aparte con la bandera del país + "Selección: N PJ · M G · P A", debajo del club/liga (desktop y mobile).
 - **Historial:** cada temporada pasada con convocatoria repite esa misma línea en su propia fila, sin desplazar las columnas de club (trofeos, OVR, stats).
 - **Resumen final de carrera:** una sección "Con la selección" con la bandera, el país y el total acumulado de partidos/goles/asistencias de toda la carrera (`construirResumenCarrera` , `career.ts:1215`) — y los trofeos de selección aparecen mezclados con los de club en la misma fila de trofeos, porque comparten el mismo array.
-

20. Banco de eventos de temporada (`data/events.ts`)

226 eventos de decisión en total, cada uno con **2 opciones** (formato completo documentado en el encabezado de `data/events.ts:1-45`):

Banco	Cantidad	Cuándo aplica
<code>generales</code>	100	Cualquier edad
<code>porEdad.novato</code>	35	Edad $\leq$ 21 años (<code>RANGO_EDAD_NOVATO_MAX</code>)
<code>porEdad.promedio</code>	35	22 a 32 años (<code>RANGO_EDAD_PROMEDIO_MAX</code>)
<code>porEdad.veterano</code>	34	33+ años
<code>altoImpacto</code>	22	Cualquier edad, máx. 1 por temporada

Selección de un evento normal (`elegirEventoPorTipo` , `career.ts:246`): para cada pausa, 50/50 si sale del banco `generales` o del banco correspondiente a la edad actual; dentro de ese banco se filtra por tipo (`"personal"` o `"deportivo"` — cada pausa siempre muestra exactamente 1 de cada). **Ningún evento se repite en la misma carrera**: se recuerda cada id ya usado (`eventosUsados` , `career.ts:122`, compartido entre bancos) y se excluye de futuros sorteos; si un banco se queda sin eventos sin usar de ese tipo (carrera muy larga), se libera el filtro para ese banco puntual antes que forzar una repetición.

Eventos de debut: 2 eventos de `porEdad.novato` (`nov-08` , `nov-32`) están escritos sobre el debut profesional en sí ("un defensor te marca en tu debut", "debutas en un estadio gigante") — un momento que ocurre una única vez. Quedan marcados con `debut: true` y `esElegibleParaDebut(evento)` (`career.ts:237`) los excluye del sorteo salvo que sea, literal, la primera pausa de decisión de toda la carrera (Temporada 1, antes de simular el primer tramo).

Eventos incompatibles con estar lesionado: 14 eventos (12 normales + `ai-16` / `ai-17` de `altoImpacto`) presuponen que el jugador está jugando en ese momento — pedir un penal, ganarse minutos, marcar al goleador rival, jugar con una molestia, recibir una crítica post-partido — algo contradictorio si está lesionado y sin sumar minutos. Quedan marcados con `noDuranteLesion: true` y `esElegibleDuranteLesion(evento)` (`career.ts:241`) los excluye del sorteo mientras `temporadaActual.lesionActiva` esté activo (ver [sección 13](#)) — el resto del banco (familia, prensa, vestuario, eventos que solo mencionan un partido próximo sin requerir que el jugador esté en cancha) sigue funcionando igual durante la baja.

Sin decisiones "gratis": cada opción de cada evento normal (no aplica a `altoImpacto` , ver el porqué abajo) tiene siempre **al menos una señal positiva y una negativa** entre `rendimiento` / `forma` / `equipo` — ninguna opción es pura-positiva ni pura-negativa, y ninguna queda neutra-plana. La idea no es que una opción "gane" a la otra en todo, sino que el jugador elija cuál le

conviene más, cuál le hace perder más o cuál le hace perder menos. Un segundo pase completo sobre las 452 opciones del banco (238 modificadas) eliminó los últimos casos de "opción comprometida en las tres dimensiones vs. opción pasiva floja" que todavía quedaban del primer ajuste — siempre inyectando la señal que falta en un eje que esa opción tenía en cero, nunca pisando la única señal que ya tenía. La única excepción a propósito sigue siendo un evento de **altoImpacto** sobre aceptar un soborno para arreglar un partido (**ai-17**) — ahí rechazar la propuesta debe ser, sí, objetivamente mejor en todo: no es un dilema de números, es una cuestión de integridad.

Eventos de Alto Impacto: sin restricción de edad, efectos mucho más fuertes (hasta ± 6 de rendimiento, ± 4 de equipo — contra $\pm 3/\pm 2$ de los eventos normales), y algunos tienen **las dos opciones en negativo a propósito** (elegir el mal menor, no "ganar"). Se identifican con un  en la tarjeta. Se sortea al crear la temporada si va a haber uno (30% de probabilidad) y en qué tramo — **career.ts:183**; si sale, reemplaza al evento del tipo que corresponda en esa pausa — mismo mecanismo de reemplazo que usa la convocatoria a la selección nacional (**sección 19**), que compite por el mismo slot "deportivo".

Cada opción define el texto del botón y sus **efectos** (**rendimiento** $-3..+3$ normal / $-6..+6$ alto impacto, **forma** nuevo estado fijo, **equipo** $-2..+2$ normal / $-4..+4$ alto impacto). También trae un texto de **resultado** en los datos — es contenido narrativo pensado para uso futuro (por ejemplo, un registro/historial de decisiones), pero **hoy no se muestra en ningún lado de la interfaz**: se sacó del toast que lo mostraba antes porque era pura redundancia con lo que ya decía el botón elegido.

efectos.fama (opcional, **EfectosOpcion**, **events.ts:71** — ver **sección 15**): a diferencia de **rendimiento** / **forma** / **equipo**, no está definido en las 226 entradas — queda en **undefined** (equivalente a 0) salvo en las **~26 entradas con personajes: ["prensa"]**, curadas a mano según el tono de cada opción (valores chicos, de -2 a $+3$). Al resolver la decisión, si la opción elegida trae **fama**, se suma a **fama.value** clampeado a **FAMA_MAX** y, si el valor efectivamente cambió, se avisa con un toast puntual ("Tu fama sube."/"Tu fama baja.", **career.ts:1110-1115**) — el resto de los eventos (familia, vestuario, deportivos) no mueve la fama, solo la prensa te hace más o menos conocido. A diferencia de **rendimiento** / **equipo** (que sí tienen su propia etiqueta en el botón antes de elegir, ver **sección 22**) y de **forma** (visible después, en la píldora de forma del panel), la fama no se previsualiza en la tarjeta — el toast es la única señal de que una decisión de prensa la movió, para que el cambio nunca quede en absoluto silencio.

Lesiones (contenido, no la lógica — ver **sección 13**): 17 lesiones reales con nombre y descripción médica, repartidas en nivel3 (6, leves), nivel2 (6, moderadas) y nivel1 (5, graves — LCA, fractura de tibia/peroné, tendón de Aquiles, hernia discal, rotura muscular grado 3).

21. Base de datos de ligas y equipos (`data/database.ts`)

29 ligas reales, cada una con sus 3 ejes ocultos de 0 a 100 (`fuerza` / `prestigio` / `economía` — ver [sección 14.0](#)). Las primeras 10 (las "grandes" originales) tienen estos valores puestos a mano desde el arranque del proyecto; las otras 19 se sumaron después en una incorporación masiva, con la misma metodología:

Liga	País	Confed.	Fuerza	Prestigio	Economía
Premier League	Inglaterra	UEFA	96	92	100
La Liga	España	UEFA	93	97	88
Serie A	Italia	UEFA	88	90	78
Bundesliga	Alemania	UEFA	87	82	82
Ligue 1	Francia	UEFA	82	75	75
Primeira Liga	Portugal	UEFA	76	80	58
Süper Lig	Turquía	UEFA	74	72	65
Brasileirão Série A	Brasil	CONMEBOL	74	80	45
Eredivisie	Países Bajos	UEFA	72	74	60
Pro League	Bélgica	UEFA	70	62	55
Primera División Argentina	Argentina	CONMEBOL	68	85	25
Liga Premier Rusa	Rusia	UEFA	66	60	55
Liga MX	México	CONCACAF	66	55	42
Primera División (Uruguay)	Uruguay	CONMEBOL	64	72	22
Liga Premier de Ucrania	Ucrania	UEFA	64	62	30
J1 League	Japón	AFC	62	55	50
Primera División (Chile)	Chile	CONMEBOL	60	62	30
Scottish Premiership	Escocia	UEFA	60	58	42
MLS	Estados Unidos	CONCACAF	58	40	55

Liga	País	Confed.	Fuerza	Prestigio	Economía
Super League Greece	Grecia	UEFA	58	55	38
Serie A (Ecuador)	Ecuador	CONMEBOL	56	50	22
Super League China	China	AFC	55	50	48
Primera División (Costa Rica)	Costa Rica	CONCACAF	52	48	25
Primera A (Colombia)	Colombia	CONMEBOL	55	50	20
Primera División (Paraguay)	Paraguay	CONMEBOL	50	54	16
Liga 1 (Perú)	Perú	CONMEBOL	48	45	18
Primera División (Venezuela)	Venezuela	CONMEBOL	44	35	16
Primera División (El Salvador)	El Salvador	CONCACAF	44	38	14
Primera División (Bolivia)	Bolivia	CONMEBOL	42	38	15

24 de las 29 tienen el campo **pais** cargado con un país que existe en **COUNTRIES** (**data/countries.ts**) — esas son las que pueden ser el punto de partida "local" en la creación de personaje (ver **sección 5**). Turquía, Grecia, Rusia, China y Ucrania tienen liga cargada pero **no** son nacionalidades elegibles todavía — se puede fichar por sus clubes, pero no arrancar la carrera como local ahí.

510 equipos reales repartidos en esas 29 ligas, cada uno con: nombre real, sus propios **fuerza** / **prestigio** / **economía** , iniciales y colores propios (para el placeholder si el escudo no carga) y el nombre del archivo de escudo real. Los ~25 clubes más reconocibles del mundo de las 10 ligas originales tienen esos 3 valores puestos a mano; el resto (incluidos los 296 equipos de las 19 ligas nuevas) se derivó con una variación estable por club (mismo id → siempre el mismo resultado) a partir de 3 niveles de referencia por liga — "grande" / "consolidado" / "humilde" — para que no todos los equipos de una misma liga terminen con el número idéntico.

Nombres cortos, "como se los conoce": los 510 equipos usan el nombre por el que se los reconoce popularmente en vez de su denominación social completa (194 renombrados) — por ejemplo "Club Sportivo Independiente Rivadavia" pasó a ser "Independiente Rivadavia", "Club Atlético Boca Juniors" a "Boca Juniors", "FC Barcelona" a "Barcelona", "Sport Club Corinthians Paulista" a "Corinthians", "FC Internazionale Milano" a "Inter". Se dejaron sin tocar los casos donde el prefijo/sufijo es parte genuina del nombre reconocido en su país (Bayern München, Borussia Dortmund, Hamburger SV, VfB Stuttgart, Club Brugge, Royal Antwerp, etc.) — no hay dos equipos con el mismo nombre visible dentro de una misma liga.

Escudos reales para toda la incorporación nueva: los 296 equipos, las 19 ligas y sus 38 trofeos (liga + copa nacional de cada una) tienen su imagen real cargada — no hay ninguna liga nueva con ícono

genérico de respaldo, salvo el trofeo de liga de Eredivisie (que sí tiene su logo real, pero todavía no un trofeo de campeón propio) y el logo de liga de Costa Rica.

Sustituciones por falta de escudo: cuando un club realmente vigente en la temporada de referencia no tenía imagen disponible, se lo reemplazó por otro club real del mismo país que sí la tenía (nunca por un club inventado) — por ejemplo, en la J1 League japonesa Mito HollyHock/JEF United Chiba/V-Varen Nagasaki se reemplazaron por Albirex Niigata/Shonan Bellmare/Yokohama FC; en la Liga Premier Rusa, Akron Tolyatti/Dynamo Majachkalá/Rodina Moscú por Nizhni Nóvgorod/Sochi/Ural Yekaterinburg. La liga de Ucrania quedó con **16 equipos reales** en vez de sus 20 oficiales de esta temporada, por no tener sustitutos disponibles para completar los 4 que faltaban — se prefirió esto antes que inventar clubes sin escudo real.

71 competencias reales ([data/database.ts](#)):

- **29 ligas domésticas** (una por cada liga cargada), con la cantidad real (o una referencia realista) de partidos de su formato vigente.
- **29 copas domésticas**, una por liga — de las 10 originales (FA Cup, Copa del Rey, Coppa Italia, DFB-Pokal, Coupe de France, Copa do Brasil, Copa Argentina, Copa México, Lamar Hunt U.S. Open Cup, Copa Colombia) a las 19 nuevas (KNVB Beker, Taça de Portugal, Croky Cup, Copa de Turquía, Scottish Cup, Copa de Grecia, Copa de Rusia, Copa del Emperador, Copa de China, Copa Perú, Copa Simón Bolívar, Copa Chile, Copa AUF Uruguay, Copa Venezuela, Copa Ecuador, Copa Costa Rica, Copa Paraguay, Copa Presidente, Copa de Ucrania).
- **7 competencias internacionales de club** por confederación/categoría: Champions League y Europa League (UEFA), Libertadores y Sudamericana (CONMEBOL), Concacaf Champions Cup (CONCACAF no tiene un segundo nivel continental vigente), y **AFC Champions League Elite/Two** (agregadas junto con J1 League y Super League China, para que sus clubes también tengan a qué clasificar — ver [sección 14](#)).
- **6 competencias de selección:** Copa del Mundo + una copa continental por confederación (Copa América, Eurocopa, Copa Oro, Copa Africana de Naciones, Copa Asiática) — ver [sección 19](#).

Todos los números de partidos (mínimos garantizados + rondas extra) son una referencia realista basada en el formato vigente de cada torneo — ver los comentarios junto a cada entrada en el archivo para el detalle de cada formato.

47 selecciones nacionales ([GameDatabase.selecciones](#)) — una por cada país de la creación de personaje, con **fuerza** / **prestigio** propios y su confederación ([UEFA](#) / [CONMEBOL](#) / [CONCACAF](#) / [CAF](#) / [AFC](#)) — ver [sección 19](#) para cómo se usan.

22. Interfaz: componentes, composables y responsive

La interfaz vanilla manipulaba el DOM a mano (funciones tipo

`crestHtml` / `animarNumero` / `capturarPosicionesCards` que devolvían o mutaban HTML como string) porque no había otra opción sin un framework. La reescritura a Vue reemplaza casi todo ese código por **componentes** (estado + template declarativo) y **composables** (lógica reutilizable con estado propio, en `src/composables/`) — el comportamiento final es intencionalmente el mismo, documentado acá con sus equivalentes nuevos.

- **Tokens de diseño** centralizados en `:root` de `assets/base.css:6` (colores, radios) — compartidos por las 3 vistas.
- **Escudos con fallback:** `CrestImg.vue` (`components/CrestImg.vue`) reemplaza a `crestHtml` / `ligaCrestHtml` / `crestFallback` — un `<img>` con `@error` que conmuta un `ref` local (`fallo`) para renderizar en su lugar un placeholder de iniciales + degradado de los colores del club. Los escudos de liga no llevan ese fondo — solo el logo (clase `team-crest--liga`).
- **Banderas reales** vía `flagcdn.com` (los emoji de bandera no se dibujan en Windows) — `FlagImg.vue` , mismo patrón de fallback que `CrestImg.vue` , con el emoji como respaldo de texto si la imagen falla.
- **Trofeos:** siluetas PNG en `assets/escudos/trofeos/` , pintadas vía `mask-image` con un dorado **propio** (`--trophy-gold: #d4af37` , `assets/base.css:21`) — el color original del archivo no importa, solo su transparencia define la forma (`TrofeoIcon.vue`). Este dorado es deliberadamente distinto del `--accent` ámbar que usan los botones y el nivel "oro" del OVR: si el trofeo reutilizara ese mismo color, se perdería entre el resto de la interfaz en vez de leerse como un logro aparte.
 - **Historial en mobile:** el nombre del trofeo va apilado debajo de su ícono (no en un listado aparte) — oculto por default, se revela al tocar la tarjeta entera de esa temporada. `TimelineList.vue` lo maneja con un `Set` reactivo de temporadas expandidas (`expandidas` , `TimelineList.vue:32`) y `.timeline-item--expandida` como clase condicional, en vez de un listener delegado sobre el DOM. En desktop el nombre completo ya está disponible al pasar el mouse (`title`).
 - `.trophy-card__name-under` (`assets/base.css:336`): `font-size: 0.35rem` — un 30% más chico que el original (0.5rem), que se leía grande al lado de los íconos de trofeo de equipo. Chico y angosto también en ancho — nombres largos ("Liga Premier Rusa", "UEFA Europa League") entrarían en 2-3 líneas con medidas más generosas, agravando el salto de altura.
 - **Revelado suave:** antes era un toggle instantáneo (`display: none / block`) — la tarjeta "saltaba" de golpe a su alto nuevo al tocarla, porque `display` no se puede animar. Ahora transiciona `max-height` (0 → `2.5em`) y `opacity` (0 → 1) en 0.3s/0.25s (`assets/base.css:336-352`), así que el texto se despliega y se repliega de forma gradual en vez de aparecer de golpe.

- **Alineación entre trofeos de equipo y premios individuales:** `.trophies` / `.trophy-card` (`assets/base.css:229` y `:235`) usan siempre `align-items: flex-start` — sin excepciones por modo ícono-solo ni por tarjeta expandida/colapsada, a diferencia de un intento anterior. Sirve para dos cosas a la vez: si el nombre de un trofeo ocupa 2 líneas (pill angosta en mobile, o el modo apilado del historial) mientras los demás quedan en 1, centrar desalinearía el ícono de esos trofeos más cortos; y alinea las CAJAS de cada ícono entre sí sin importar su tamaño real, que es donde entran los premios individuales.
 - Los premios individuales usan una caja de ícono más chica que los trofeos de equipo en modo píldora (`.trophy-card__icon-img--premio` , `assets/base.css:301`: 0.8rem contra 1rem) — ahí no hay problema, porque no comparten fila con otro ícono más que el texto, ya resuelto por `flex-start` . Pero en modo ícono-solo (historial), esa misma reducción de caja (antes 1.7rem contra 2.1rem) hacía que, con `flex-start` , el borde de ABAJO de la caja de un premio quedara más arriba que el de un trofeo de equipo — los íconos se veían "subidos" al lado de una copa. La caja de un premio en icon-only **ya no se achica** (`assets/base.css:320`): queda del mismo 2.1rem que un trofeo de equipo, y en cambio se achica el `mask-size` (la silueta renderizada en sí) a 1.7rem dentro de esa caja pareja — con todas las cajas del mismo alto, `flex-start` ya alinea tanto los bordes de arriba como los de abajo, sin necesitar ningún caso especial por tamaño de ícono.
 - Eso solo no alcanzaba: las siluetas de trofeo de equipo vienen dibujadas "apoyadas" bien abajo de su propio archivo PNG, con bastante margen transparente arriba (así se ven las copas reales, angostas y con base), mientras que las de los premios individuales casi no tienen margen y ocupan casi todo el archivo. Con `mask-position: center` (el default, `assets/base.css:262`) cada silueta queda centrada según SU PROPIO margen interno — como esos márgenes son bien distintos entre categorías, las copas terminaban flotando bastante más abajo que los premios dentro de su propia caja, aun con las cajas ya parejas. `.trophy-card--icon-only .trophy-card__icon-img` (`assets/base.css:290`) fija `mask-position: bottom` para apoyar cada silueta contra el borde inferior de su caja, sin importar ese margen interno — acotado a icon-only nada más: en el modo píldora el ícono es chico (1rem/0.8rem) y el mismo cambio hacía que la silueta colgara visiblemente más abajo que el texto, así que ahí se mantiene `center` .
 - `.trophy-card--stacked` (`assets/base.css:337`, nombre apilado debajo del ícono — historial de mobile expandido, ver arriba), al usar `flex-direction: column` , necesita revertir a `align-items: center` explícito — con `column` , `align-items` pasa a controlar el eje horizontal, así que heredar `flex-start` habría dejado el ícono pegado a la izquierda en vez de centrado arriba del nombre. La captura que genera el botón "  " no se ve afectada por ninguno de estos casos — el `<canvas>` dibuja el texto de cada trofeo en una sola línea siempre (`fillText` no hace wrap) y no usa `mask-image` , así que nunca tuvo ninguno de los dos desalineamientos.
- **Premios individuales más chicos:** las siluetas de Bota de Oro/Balón de Oro/Once Ideal son visualmente más "llenas" que las copas de trofeos de equipo — al mismo tamaño de caja se leían más grandes al lado de esas. `TrofeoIcon.vue` detecta los 3 por nombre de archivo y les agrega la clase `trophy-card__icon-img--premio` , achicada en los dos contextos de tamaño

(pill: caja más chica; icon-only: mask-size más chico dentro de la misma caja, ver el punto de arriba).

- **Color del badge de OVR** (`ovrTierColor` , `game/format.ts:8`) — 6 niveles fijos, de metal a gema, proporcionales al rango real de carrera (45–99):

OVR	Color
45–65	Bronce
66–79	Plata
80–89	Oro
90–92	Zafiro
93–95	Rubí
96–99	Amatista

- **Etiquetas de efecto en cada opción de decisión**
(`efectoRendimientoTexto` / `efectoEquipoTexto` , `components/carrera/DecisionCardItem.vue:13-18`): antes de elegir, cada botón muestra de forma explícita qué le va a pasar a tu rendimiento, tu forma y al equipo si lo tocas — no hay efectos ocultos en las decisiones de evento.
- **Tarjeta de "Fin de carrera" (retiro forzoso por edad)**: ocupa toda la fila y va centrada (`.decision-card--retiro-forzoso` , aplicada desde `OfertaCardItem.vue` y estilada en `views/CarreraView.vue:429-445`) para que se sienta un momento aparte, más solemne — pero ese centrado heredado también dejaría el nombre del club y su liga centrados dentro del bloque escudo+texto, desalineados del escudo que va al lado (en vez de leerse junto a él como en el resto de tarjetas). `.decision-card__team` fuerza `text-align: left` de vuelta, solo para ese bloque — el párrafo de despedida sigue centrado.
- **Animaciones de tramo**: los números del spotlight (partidos, goles, OVR, anillo de progreso) no saltan de golpe — se animan con un *ease-out* cúbico durante 900ms vía el composable `useAnimatedNumber` (`composables/useAnimatedNumber.ts:16` , duración = `GameConfig.ANIMACION_TRAMO_MS`), usado tanto por el bloque desktop como por el mobile de `SpotlightCard.vue` — un único componente renderiza ambos markups (alternados por CSS, no por JS) en vez de que cada uno tenga su propia función de animación.
- **Reacomodo de tarjetas al resolver una decisión**: cuando queda una tarjeta menos en la misma pausa, la que sigue no salta de golpe a su nueva posición — `DecisionsPanel.vue` envuelve el carrusel en un `<TransitionGroup name="decision-card">` (`DecisionsPanel.vue:93`), que aplica FLIP automáticamente vía Vue en vez de la técnica manual (`capturarPosicionesCards` / `animarReacomodoCards`) del original — la clase `.decision-card-move` fija la curva del desplazamiento en 550ms con `cubic-bezier(0.4, 0, 0.2, 1)` (`views/CarreraView.vue:627-629`). **Solo en desktop**: en mobile, trasladar la tarjeta (FLIP mueve en X)

entra en conflicto con el scroll-snap nativo del carrusel de decisiones — `.decision-card-move { transition: none }` lo desactiva puntualmente dentro del media query mobile (`views/CarreraView.vue:669`), dejando solo el fundido de entrada/salida (`.decision-card-enter-active / -leave-active` , 250ms) sin trasladar nada.

- **Lesión activa — efecto de luz roja:** mientras el jugador tiene una lesión en curso, la tarjeta de spotlight de la temporada (desktop y su equivalente mobile, ambos dentro de `SpotlightCard.vue`) muestra un borde y resplandor rojo (`.spotlight-card--lesionado / .spotlight-mobile--lesionado` , `SpotlightCard.vue:173`) — el mismo lenguaje visual que ya usaban la tarjeta de evento de alto impacto y el ícono de mundo de la convocatoria a la selección, para que "algo importante está pasando" se lea igual en toda la interfaz. Es reactivo al estado de `temporadaActual.lesionActiva` del store, así que se repinta apenas se diagnostica la lesión, no recién al simular el próximo tramo.
- **Línea de diseño móvil independiente:** por debajo de los 640px, cada componente de carrera (`SpotlightCard.vue` , `TimelineList.vue` , `HeroPanel.vue`) renderiza su propio bloque HTML más chato dentro del mismo archivo — alternado con el de desktop vía CSS (`display: none / flex` en el media query), no generado aparte por JS — y el panel de decisiones pasa a un carrusel de una tarjeta a la vez con scroll-snap, sin JavaScript adicional para eso.
- **Tarjeta para compartir el resumen de carrera:** el botón " 📄 " en la cabecera de `ResumenModal.vue` (`ResumenModal.vue:86`) genera una imagen propia con los mismos datos del resumen — no es una captura del modal (eso pediría una librería externa que el proyecto no usa), es una tarjeta de 1080px de ancho dibujada a mano en un `<canvas>` (`generarTarjetaResumenCanvas` , `game/resumen-canvas.ts`): el logo real del juego (`assets/logo/logo_leyenda_transparent.png` , 70px de alto) en la esquina, escudo del último club, degradado con sus colores, badge de pico de OVR, gráfico de evolución de OVR, grid de estadísticas, recorrido de clubes, sección "Con la selección" (bandera + país + partidos/goles/asistencias, si aplica) y trofeos. Tres niveles de respaldo según lo que soporte el navegador (`ResumenModal.vue:48-78`): Web Share API con archivo (abre el selector nativo — ideal en mobile) → Clipboard API (`navigator.clipboard.write` , lo más práctico en desktop) → `window.open` como último recurso.
 - **Alto dinámico:** el recorrido de clubes y los chips de trofeos pasan a una fila/línea nueva cuando no entran en el ancho disponible (una carrera larga puede tener 7+ clubes) — nunca se dibujan todos en una sola fila fija, así que los escudos de más nunca quedan fuera de la tarjeta. Antes de dibujar nada se mide cuántas filas va a necesitar cada sección variable y se fija el alto real del canvas en base a eso (`H` , `resumen-canvas.ts:262`: 1350px o 1450px si hubo convocatorias, más lo que sumen las filas extra) — nunca un número fijo.
 - **Nitidez de los escudos:** por defecto el canvas reescala imágenes con `imageSmoothingQuality: "low"` (pensado para animaciones a 60fps, no para una sola exportación estática) — con escudos fuente de 1500×1500px, de sobra para verse nítidos, esto los dejaría borrosos al reducirlos a ~84-130px. Se fija en `"high"` justo después del último resize del canvas (`resumen-canvas.ts:270` — cambiar `width / height` resetea todo el estado del contexto, así que tiene que ir después, no antes).

- **Imágenes cross-origin en el canvas:** la bandera del país sale de `flagcdn.com` (`GameConfig.RUTA_BANDERAS`), un origen distinto al del juego. Dibujar una imagen así en el canvas sin marcarla `crossOrigin = "anonymous"` lo deja "tainted" (contaminado) y el navegador bloquea después cualquier intento de exportarlo (`toBlob` / `toDataURL`) con un `SecurityError` — rompería la tarjeta entera apenas la carrera incluyera convocatorias a la selección. La bandera se carga con `cargarImagenSeguraCrossOrigin` (`resumen-canvas.ts:27`) en vez de la función genérica `cargarImagenSegura` (reservada para assets propios del sitio, `resumen-canvas.ts:14`); si el servidor remoto no coopera con CORS, cae sola al respaldo de emoji sin romper nada.
- **Chips de trofeo con la silueta real** (`dibujarIconoTrofeo` , `resumen-canvas.ts`): cada chip dibuja el mismo ícono real que ya usa la interfaz (`RUTA_ESCUDOS_TROFEOS`), no el 🏆 genérico — el canvas no tiene `mask-image` , así que el mismo efecto de "recolorear una silueta según su alpha" se logra dibujando la imagen en un `<canvas>` auxiliar en blanco y pintando encima con `globalCompositeOperation: "source-atop"` , y recién ese resultado ya recoloreado se copia al canvas principal. Hacer el recoloreado directo sobre el canvas principal (encima del fondo translúcido del chip, ya dibujado) pinta el cuadrado entero en vez de solo la silueta, porque `source-atop` ve ese fondo como "ya no transparente" en todo el recuadro. Si el trofeo todavía no tiene imagen cargada, ese chip cae solo al 🏆 + texto, igual que en la interfaz.
- **Con la selección:** cuando hubo convocatoria esa temporada, el spotlight y el historial muestran una línea aparte con la bandera del país + partidos/goles/asistencias con la selección (ver [sección 19](#)) — nunca mezclada con los números de club.
- **Chips del hero** (edad, país, valor de mercado): los 3 comparten el mismo estilo neutro (texto blanco, borde translúcido) en `HeroPanel.vue` — visualmente el mismo tipo de dato, sin que ninguno destaque sobre los otros dos sin motivo.
- **Chip de fama:** entre el chip de país y el de valor de mercado, `HeroPanel.vue` agrega un 4to chip con `famaTierLabel(career.fama)` — a propósito **no** comparte el estilo neutro de los otros 3: usa `famaTierColor` como color de texto/borde, la misma paleta de 6 niveles descrita en [sección 15](#), para que se lea de un vistazo como un dato distinto (reputación, no un stat de cancha).
- **Badge de Capitán:** `.lineup-tag.lineup-tag--capitan` ("Capitán", color `#facc15`) en `SpotlightCard.vue` — mismo lugar y mismo estilo de pill que ya usan los badges de Titular/Suplente/Préstamo, `v-if="s.capitan"` , desktop y mobile. En el historial, `TimelineList.vue` agrega un sufijo `(C)` después del nombre del club de esa temporada, mismo patrón que ya usa `(préstamo)` .
- **Reseteo de zoom en iOS** (`resetZoom` , `composables/resetZoom.ts`): el original cambiaba de pantalla con una carga de página real, que resetea gratis cualquier zoom que el navegador haya dejado activo (típicamente al enfocar un input con `font-size` chico — varios campos del formulario de personaje, y el número del `NumeroModal` , tienen menos de 16px). En la SPA, sin recarga real, ese zoom se arrastraría de una vista a la siguiente — se fuerza el reseteo tocando `maximum-scale` del `<meta name="viewport">` un instante y devolviéndolo, con tres disparadores distintos:

- `router.afterEach` (`router/index.ts:54`) para cualquier navegación.
- Una llamada explícita como **lo primero** que hace el `onMounted` de cada una de las 3 vistas (`PersonajeView.vue`, `EquipoView.vue`, `CarreraView.vue`) — sin esa llamada explícita, una medición de layout hecha por la vista nueva (ver el punto siguiente) podía correr antes de que el reseteo del router surtiera efecto y quedar calibrada contra un zoom todavía "fantasma".
- `instalarResetZoomAlCerrarTeclado()` (`composables/resetZoom.ts:44`): un único listener global de `focusout`, instalado una sola vez en `onMounted` de `App.vue`, que llama a `resetZoom()` cada vez que se blurea cualquier `<input>` / `<textarea>` de la app. Cubre el caso que los otros dos disparadores no cubrían: cerrar el teclado tocando afuera del campo (o "listo" del teclado) sin navegar ni cerrar un modal — antes ese zoom se quedaba pegado hasta la próxima navegación. `NumeroModal.vue` ya no llama a `resetZoom()` por su cuenta en `cerrar()` — su `(document.activeElement as HTMLElement | null)?.blur()` dispara este listener global de forma síncrona, así que la llamada propia había quedado redundante (y, de hecho, disparaba el reseteo dos veces seguidas).
- **Altura real de viewport en mobile (`--vh-real`)**: el layout de `CarreraView.vue` (hero fijo / centro scrolleable / footer de decisiones fijo) depende de conocer la altura visible real de la pantalla. `100dvh` la calcula bien en Safari/iOS, pero varios navegadores mobile (Chrome/Firefox en Android, algunos in-app browsers) la calculan mal al cargar la página y dejan una franja del footer tapada. `actualizarAlturaViewport()` (`views/CarreraView.vue:39`) prioriza `window.visualViewport?.height` sobre `window.innerHeight` — esa API está pensada específicamente para "cuánto se ve de verdad ahora" y descuenta el teclado en pantalla y los ajustes de la barra de direcciones, mientras que `innerHeight` dejaba casos de layout cortado en Android; `innerHeight` queda como respaldo para navegadores sin `visualViewport`. Se mide por JS al montar el componente (después de `resetZoom()`, ver arriba) y en cada `resize/orientationchange` (incluido el propio evento `resize` de `visualViewport`), y esa variable pisa a `100dvh` en `.career-shell` como última palabra (`views/CarreraView.vue:180`) — `100vh` y `100dvh` quedan como respaldo en cascada para cuando el JS todavía no corrió. En Android la barra de direcciones puede seguir animando/asentándose un instante después del montado inicial, sin disparar ningún evento de `resize` (no hay scroll de página real que la oculte) — una remediación corta 300ms después (`reintentoAlturaViewport`, `views/CarreraView.vue:85`) agarra el valor ya asentado por si la primera quedó de más, y se limpia con `clearTimeout` al desmontar.
- **Remediación en cada cambio de pausa, no solo al montar**: en Android + Edge se reportó el panel de decisiones quedando fuera del área visible justo al cerrar una temporada — la barra dinámica de Edge (a diferencia de Chrome, con quien se probó originalmente este fix) no siempre dispara `resize` al reaccionar a una interacción cerca del borde inferior de la pantalla (justo donde viven los botones de decisión), así que `--vh-real` quedaba con un valor viejo y el panel de decisiones (hermano flex de `.career`, no una tarjeta aparte — ver **sección 22** más abajo) terminaba calculado sobre una altura mayor a la real. Dos cambios: `visualViewport.addEventListener('scroll', ...)` (`views/CarreraView.vue:79`) — ese evento se dispara cuando la barra del navegador se muestra/oculta y desplaza el viewport visual, incluso sin un `resize` de por medio — y un `watch` sobre ``${numero}-${checkpointIndex}-${carreraFinalizada}`` de la temporada actual

(`views/CarreraView.vue:109-114`) que remide con `nextTick` cada vez que cambia la pausa (cierre de temporada, nueva ventana de fichajes, retiro) — el momento de mayor reflow de toda la vista, y el más probable para que el navegador reaccione con su propia barra sin que el código se entere.

- **Toast:** composable compartido `useToast` (`composables/useToast.ts:9`) reemplaza a `showToast` — antes cada una de las 3 pantallas tenía su propia copia del mismo patrón (mensaje + visible + timer) escrita a mano; ahora cada vista lo instancia una vez (`message` / `visible` / `show`) y, en `CarreraView.vue`, además usa `drainQueue` para vaciar la cola de mensajes que produce el store (`career.mensajes`, el reemplazo reactivo de los `showToast( ... )` sueltos que el motor original llamaba directo). En `CarreraView.vue` el toast aparece debajo del hero en vez de abajo de la pantalla (`.body--career .toast--visible`, `views/CarreraView.vue:232`), porque ahí abajo siempre está el panel de decisiones. En mobile (`@media (max-width: 640px)`) ocupa casi todo el ancho de pantalla (`calc(100vw - 1.5rem)`) en vez de ajustarse solo al texto — más fácil de leer en una pantalla chica.
 - **Pie de versión** (`GameConfig.VERSION`, `GameConfig.FECHA_PUBLICACION` — `config.ts:389-390`): un único punto de verdad para el número de versión y la fecha de publicación. Cada una de las 3 vistas (`PersonajeView.vue`, `EquipoView.vue`, `CarreraView.vue`) renderiza su propio `<footer class="app-footer">` leyendo esas mismas dos constantes — en `PersonajeView.vue` / `EquipoView.vue` es el último elemento de la vista (scroll normal); en `CarreraView.vue` va dentro de `.career`, después del historial, para no restarle alto fijo al hero/spotlight/decisiones.
-

23. Persistencia y estado

A diferencia de la versión vanilla (que solo guardaba la identidad/club inicial del jugador y perdía toda la progresión de la carrera al recargar), la reescritura a Vue agrega **guardado real de partida en curso** — recargar la página, o cerrar la pestaña y volver más tarde, retoma la carrera exactamente donde quedó.

- `localStorage["leyenda-carrera"]` (`STORAGE_KEY` , `career.ts:38`) guarda un snapshot serializado con todo el estado necesario para reconstruir la carrera: `player` , `temporadaActual` , `temporadasFinalizadas` , `temporadasEnClubActual` , `esPrimerClub` , `edadRetiroForzoso` , `factorTalento` , `potencialTecho` , `carreraFinalizada` , `eventosUsados` (como array — se reconstruye como `Set` al cargar) y el contexto de la solicitud de cambio de dorsal.
 - `guardar()` (`career.ts:1281`) se llama después de **cada** checkpoint que cambia el estado de la carrera: iniciar carrera, cerrar un tramo, cerrar una temporada, resolver una decisión o una oferta, confirmar un cambio de dorsal, y retirarse — nunca hay una ventana donde el progreso en memoria esté más adelantado que lo último guardado. Envuelto en `try/catch` sin re-lanzar: si `localStorage` falla (modo privado, cuota llena), la carrera sigue jugable en memoria, simplemente no persiste ese guardado puntual.
 - `cargar()` (`career.ts:1316`) reconstruye todo el estado del store a partir del snapshot guardado, y devuelve `true` / `false` según si había una partida guardada válida. Se invoca **solo** al entrar a `/carrera` sin una carrera ya activa en memoria (`views/CarreraView.vue:44`) — o sea, al recargar la página estando en esa ruta (la navegación normal Personaje → Equipo → Carrera arranca la carrera directo en memoria vía `iniciarCarrera` , sin pasar por `cargar()`). Si tampoco hay nada guardado, redirige a `/` .
 - `hayCarreraGuardada()` (`career.ts:1307`) y `limpiarPartidaGuardada()` (`career.ts:1357`) están expuestas por el store (comprobar si existe una partida sin cargarla, y borrarla del todo) pero **ninguna vista las usa todavía** — quedan disponibles para una futura pantalla de "continuar carrera" o "borrar mi progreso" en la creación de personaje.
 - `localStorage["leyendaPlayerDraft"]` sigue existiendo aparte, con el mismo rol que tenía en la vanilla: identidad del jugador + club/OVR inicial, desde la creación de personaje hasta que `iniciarCarrera` arranca la Temporada 1 (ver [sección 4](#) y [sección 5](#)) — no se toca una vez la carrera está en curso.
 - No hay backend, base de datos externa, ni llamadas de red propias del juego (aparte de pedir escudos/banderas/imágenes de trofeos como archivos estáticos, y las banderas de país a `flagcdn.com`).
-

24. Tabla completa de constantes de balance

Todas viven en `app/src/game/config.ts`. Cambiar cualquiera de estos números es la forma correcta de recalibrar el juego — nunca hay "números mágicos" repetidos sueltos en otros archivos.

Constante	Valor	Qué controla
<code>EDAD_MIN</code> / <code>EDAD_MAX</code>	16 / 19	Rango de edad al crear personaje
<code>DORSAL_INICIAL_PROB_BAJO</code> / <code>_MEDIO</code>	0.7 / 0.2	Reparto por bandas del dorsal inicial: 70% 1-30, 20% 31-50, 10% (resto) 51-99
<code>RANGO_EDAD_NOVATO_MAX</code>	21	Techo de edad para el banco de eventos "novato"
<code>RANGO_EDAD_PROMEDIO_MAX</code>	32	Techo de edad para el banco "promedio" (arriba, "veterano")
<code>EJE_MAX</code>	100	Techo de la escala oculta de los 3 ejes de clasificación (fuerza/prestigio/economía) de cada club, liga y selección — el piso es 0, implícito en cada <code>clamp</code>
<code>PODER_PESO_FUERZA</code> / <code>_PRESTIGIO</code> / <code>_ECONOMIA</code>	0.4 / 0.35 / 0.25	Peso de cada eje al combinarlos en el "poder" único (OVR inicial, ventana de ofertas, objetivo)
<code>ECONOMIA_PISO_LIGA</code>	0.6	La economía de un club nunca cae debajo de este % de la de su propia liga
<code>VALOR_PESO_ECONOMIA</code> / <code>_PRESTIGIO</code>	0.6 / 0.4	Peso de economía vs. prestigio en el valor de mercado (no usa el eje fuerza)
<code>OVR_INICIAL_MIN</code> / <code>MAX</code>	50 / 65	Rango de OVR con el que puede arrancar un novato
<code>OVR_PESO_EQUIPO</code> / <code>OVR_PESO_LIGA</code>	0.55 / 0.45	Peso de club vs. liga al combinar sus "poder"/"valorScore"
<code>OVR_SUERTE_VARIACION</code>	±4	Variación aleatoria del OVR inicial
<code>VALOR_MERCADO_BASE</code>	3,000,000	Valor de mercado en el piso absoluto de OVR
<code>VALOR_MERCADO_CRECIMIENTO</code>	1.08	Multiplicador de valor por cada punto extra de OVR

Constante	Valor	Qué controla
VALOR_MULTIPLICADOR_CLUB_MIN / MAX	0.5 / 1.4	Rango del multiplicador de valor según economía/prestigio combinados del club/liga
OFERTA_VARIACION_VALOR	±8%	Variación cosmética del valor mostrado en cada oferta
OFERTA_UMBRAL_CAIDA_VALOR	0.4	Mínimo % de tu valor actual que debe ofrecerte un club para tener sentido
OFERTA_TOP_ENCAJE_FRACCION	0.4	% superior (por cercanía a tu poder objetivo) del que se sortean las ofertas
OFERTA_TOLERANCIA_OVR	13	Ancho de la "ventana de OVR" de cada club (±)
CATEGORIA_EQUIPO_PERCENTIL	humilde 0-50% / consolidado 50-85% / grande 85-100%	Percentiles de poder dentro de la liga, para las ofertas iniciales (sección 5)
PESO_ESCALA_DISTANCIA	10	Divisor que lleva la distancia de poder (0-100) a una escala chica antes del exponente
PESO_DISTANCIA_LIGA	0.6	Cuánto pesa desviarse en liga vs. en equipo al elegir ofertas
PESO_EXPONENTE	2.2	Qué tan fuerte castiga la distancia al poder objetivo
PESO_FACTOR_SOBRAR	0.05	Factor que aplica la distancia (en vez del 100%) cuando un club/liga "sobra" de poder en vez de quedar corto — ver sección 16.6
PROB_OFERTA_NOSTALGICA	0.3	Probabilidad, por ventana, de que aparezca 1 oferta de "vuelta a casa" para un veterano cuyo país de origen ya no llega a su tier de poder
EDAD_RETIRO_OFERTA	36	Desde cuándo puedes elegir retiro voluntario
EDAD_RETIRO_FORZOSO_MIN / MAX	41 / 45	Rango del que se sortea la edad de retiro forzoso (una vez por carrera)

Constante	Valor	Qué controla
EDAD_RETIRO_TRANSICION	2	Temporadas antes del retiro forzoso en las que el cupo de ofertas ya se reduce a 1
TEMPORADAS_GRACIA_CONTRATO	2	Temporadas de gracia antes de que tu club pueda "no renovarte" (clubes fichados después del primero)
TEMPORADAS_GRACIA_CONTRATO_PRIMER_CLUB	4	Igual, pero para el primer club de la carrera (el de la creación de personaje) — el doble de margen
CONTRATO_RENDIMIENTO_SALVAVIDAS	7.5	Promedio de rating de la temporada anterior que salva el contrato aunque el OVR no llegue a la ventana del club
EDAD_POTENCIAL_BONUS_MAX	8	Bono máx. de "potencial" para un jugador de 17 años
EDAD_POTENCIAL_BONUS_HASTA	24	Edad desde la que el bono de juventud llega a 0
EDAD_POTENCIAL_PENALIZACION_DESDE	30	Edad desde la que empieza la penalización de potencial
EDAD_POTENCIAL_PENALIZACION_TASA	0.7	Penalización de potencial por año, desde esa edad
EDAD_OCASO_RETORNO_PAIS	33	Edad desde la que la garantía de "entorno" prioriza tu país en vez de tu liga
TOTAL_TRAMOS_TEMPORADA	3	Bloques de partidos simulados por temporada
PROPENSION_GOL_POSICION / PROPENSION_ASISTENCIA_POSICION	ver sección 10	Probabilidad base de gol/asistencia por posición individual (12 posiciones)
GRUPOS_POSICION	ver sección 10	Agrupar las 12 posiciones en 5 grupos (arquero/central/lateral/medio/ataque) — se mantiene solo para el peso de candidatos a premio (sección 14.2), ya no para la propensión de gol/asistencia
ESTADISTICAS_OVR_BASE / _EXPONENTE / _RANGO	0.22 / 1.9 / 2.78	Curva de escalado de estadísticas por OVR (factor 0.22 en el piso de carrera, 3.0 en el techo; punto neutral ×1 en ~OVR 72-73)

Constante	Valor	Qué controla
FACTOR_LIGA_COEFICIENTE	0.024	Cuánto empuja el factor de estadísticas por cada punto de diferencia entre tu OVR y el nivel que "espera" la liga/selección
FACTOR_LIGA_MIN / MAX	0.35 / 2.2	Clamp del factor de competitividad de liga (ver sección 10)
PROB_SEGUNDO_GOL_FACTOR / _TERCER_GOL_FACTOR	0.4 / 0.18	Probabilidad (relativa a probGol) de que un gol se convierta en doblete/hat-trick en el mismo partido
PROBABILIDAD_MVP_BASE	0.09	Probabilidad base de MVP por partido
BONUS_MVP_POR_GOL / _ASISTENCIA	0.14 / 0.08	Bono de probabilidad de MVP por cada gol / por asistencia en el partido
BONUS_RATING_POR_GOL / _ASISTENCIA	0.7 / 0.4	Bono de rating por cada gol / por asistencia en ese mismo partido
RATING_BASE	6.5	Nota de un partido neutral (factor $\times 1$, sin gol ni asistencia)
RATING_FACTOR_COEFICIENTE	1.3	Cuánto empuja el factor de rendimiento la nota para arriba o para abajo (antes 2.5 — ver sección 10)
OVR_TRAMO_BASE	0.7	Crecimiento natural de OVR por tramo
OVR_TRAMO_RENDIMIENTO_DIVISOR	6	Cuánto divide el rendimiento acumulado antes de sumarse al crecimiento
OVR_TRAMO_VARIACION_MIN / MAX	-1 / +3	Variación normal de OVR por tramo (sin declive por edad)
OVR_TRAMO_DECLIVE_VARIACION_MIN	-10	Piso de variación por tramo una vez que el declive por edad ya actúa
OVR_CARRERA_MIN / MAX	45 / 99	Piso y techo absolutos de OVR durante la carrera
OVR_EDAD_PRIME_MAX	28	Hasta qué edad el crecimiento es pleno
OVR_EDAD_DECLIVE_MAX	34	Edad en la que el freno de crecimiento llega a su mínimo (fin de la meseta)
OVR_EDAD_FACTOR_MIN	0.15	Ritmo de crecimiento mínimo en el ocaso (nunca llega a cero del todo)

Constante	Valor	Qué controla
OVR_EDAD_DECLIVE_INICIO	30	Edad desde la que empieza el desgaste natural (superpuesto a la meseta)
OVR_EDAD_ACELERA_DECLIVE	37	Edad desde la que el desgaste se acelera
OVR_EDAD_DECLIVE_TASA_BASE / _ACELERADA	0.06 / 0.22	OVR perdido por tramo, por año, antes/después de acelerar
TALENTO_MIN / MAX	0.85 / 1.2	Multiplicador de talento oculto por carrera (acelera el crecimiento, atenúa el desgaste, y desde esta versión también pesa en <code>simularTramo</code> — ver sección 11.2)
UMBRAL_CRECIMIENTO_ACELERADO	72	OVR por debajo del cual aplica el "salto de calidad" del arranque de carrera (solo en Prime, ver sección 11.3)
CRECIMIENTO_ACELERADO_FACTOR_MAX	1.8	Multiplicador de crecimiento en el piso de ese rango (OVR 45), decreciendo a 1x en el umbral
POTENCIAL_TECNO_PROB_BAJA / _MEDIO	0.3 / 0.5	Probabilidad de sortear un techo de potencial bajo (70-79) / medio (80-85) — el resto (20%) es alto (86-98)
POTENCIAL_TECNO_FACTOR_MIN	0.08	Fracción de lo que un tramo se pasaría del techo que se deja pasar igual
FORMA_CALIDAD	ver sección 12	Calidad aportada por cada estado de forma a la fuerza de campaña
FORMA_PESO_ACUMULACION	0.5	Fracción del camino hacia el objetivo de forma que se recorre por decisión (ver sección 12)
PESO_TITULAR_INICIAL	0.4	<code>pesoTitular</code> de arranque (novato o recién fichado)
PESO_TITULAR_MIN / MAX	0.05 / 0.95	Piso y techo de <code>pesoTitular</code>
PESO_TITULAR_RATING_NEUTRO	6.5	Rating de tramo que ni suma ni resta <code>pesoTitular</code>
PESO_TITULAR_AJUSTE_RATING	0.05	Sensibilidad del ajuste de <code>pesoTitular</code> al rating del tramo
PESO_TITULAR_AJUSTE_MIN / MAX	-0.08 / 0.12	Rango del ajuste de <code>pesoTitular</code> por rendimiento en un tramo

Constante	Valor	Qué controla
PESO_TITULAR_CASTIGO_SIN_MINUTOS	-0.05	Ajuste de pesoTitular si no jugaste ningún partido ese tramo
PESO_TITULAR_CASTIGO_LESION	-0.03	Ajuste de pesoTitular si estuviste lesionado ese tramo
FUERZA_PESO_CLUB / _FORMA / _EQUIPO_ACUMULADO / _RENDIMIENTO_JUGADOR	0.4 / 0.15 / 0.2 / 0.25	Pesos de la fórmula de fuerza de campana — copa nacional, avanzar de ronda, clasificación internacional (el eje club usa solo fuerza , no el poder combinado; ver sección 14.1)
FUERZA_TITULO_PESO_CLUB / _FORMA / _EQUIPO_ACUMULADO / _RENDIMIENTO_JUGADOR	0.85 / 0.04 / 0.05 / 0.06	Pesos de la fórmula de fuerza de título — ganar la liga, ganar la final de una copa internacional (el club pesa mucho más que en la fuerza de campana)
VENTAJA_DOMINANCIA_DOMESTICA	Bayern München / PSG: +0.08 cada uno	Bonus fijo a fuerzaTitulo , solo para el título de liga — corrige clubes cuya brecha real de fuerza contra su rival doméstico es demasiado chica (ver sección 14.1)
FUERZA_RENDIMIENTO_PROMEDIO_PISO / _RANGO	6.0 / 3.0	Normaliza el promedio de rating del jugador a 0-1, usado por ambas fuerzas
PREMIOS_CANDIDATOS_N	24	Candidatos fantasma generados por temporada para los premios mundiales — ver sección 14.2
PREMIOS_OVR_MIN / _MAX	82 / 99	Rango de OVR (sesgado al centro) de los candidatos a premio
PREMIOS_PESO_GRUPO	ataque 55% / medio 25% / lateral 12% / central 8%	Reparto de grupo de posición entre los candidatos (nunca arquero)
BALON_ORO_PESO_RATING / _GOLEADOR / _TROFEOS	0.4 / 0.35 / 0.25	Pesos del puntaje combinado del Balón de Oro
BALON_ORO_REFERENCIA_GOLES	40	Goles + asistencias de una temporada de ensueño, para normalizar calidadGoleador a 0-1
ONCE_IDEAL_MARGEN_PROMEDIO	0.4	Tolerancia contra el mejor promedio del pool en tu posición, para no depender del empate exacto en el tope de rating

Constante	Valor	Qué controla
UMBRAL_CLASIFICA_PRIMER_NIVEL / _SEGUNDO_NIVEL	0.72 / 0.45	Umbrales de fuerza para clasificar a competición internacional
UMBRAL_OVR_CONVOCATORIA_BASE / _FACTOR	50 / 0.35	Fórmula del OVR de referencia (50/50 de convocatoria) según la fuerza de tu selección — ver sección 19
PROB_CONVOCATORIA_PENDIENTE_OVR	0.04	Cuánto sube/baja la probabilidad de convocatoria por cada punto de OVR de diferencia con el umbral
PROB_CONVOCATORIA_MIN / MAX	0.03 / 0.95	Piso y techo de probabilidad de convocatoria
SELECCION_PESO_JUGADOR	0.15	Cuánto empuja tu forma del momento a la "calidad" de campaña de tu selección (el resto es la fuerza fija del país)
PARTIDOS_AMISTOSO_SELECCION_MIN / _MAX	3 / 5	Partidos de una ventana FIFA sin torneo grande en juego
PARTIDOS_ELIMINATORIAS_MIN / _MAX	6 / 10	Partidos de la campaña de eliminatorias, se clasifique o no
PARTIDOS_FASE_DE_GRUPOS_SELECCION	3	Partidos garantizados de fase de grupos, ya clasificado
FORMA_BONUS_PARTICIPACION	ver sección 9	Bono/malus de participación por estado de forma
PARTICIPACION_BASE	0.65	Probabilidad base de jugar un partido
PARTICIPACION_BONUS_TITULAR	0.2	Bono si eres titular ese tramo
PARTICIPACION_OVR_REFERENCIA	55	OVR de referencia (neutral) para la probabilidad de jugar
PARTICIPACION_OVR_PESO_BAJO / _ALTO	0.08 / 0.02	Sensibilidad asimétrica al OVR: pesa más quedarte por debajo del OVR de referencia que superarlo (ver sección 9)
PARTICIPACION_MIN / PARTICIPACION_MAX	0.15 / 0.92	Piso y techo de probabilidad de jugar
PARTICIPACION_EDAD_NOVATO_PLATEAU_HASTA / _HASTA	19 / 24	Rango de edad de la penalización de novato a la participación: se mantiene al máximo hasta _PLATEAU_HASTA , luego tapering lineal a 0 en _HASTA (ver sección 9)

Constante	Valor	Qué controla
PARTICIPACION_PENALIZACION_NOVATO_MAX	0.35	Penalización máxima a la participación por ser un novato de 16-19 años, más allá de tu OVR
PARTICIPACION_PESO_RENDIMIENTO_REAL	0.15	Cuánto pesa tu promedio de rating REAL de la temporada (no solo OVR/forma/decisiones) sobre tus minutos del próximo tramo
TITULAR_OVR_REFERENCIA / TITULAR_OVR_PESO	55 / 0.02	OVR de referencia y sensibilidad de calcularTitular (ver sección 9)
TITULAR_EDAD_NOVATO_PLATEAU_HASTA / _HASTA	19 / 24	Rango de edad de la penalización de novato a la titularidad: se mantiene al máximo hasta _PLATEAU_HASTA , luego tapering lineal a 0 en _HASTA — constantes propias, no compartidas con PARTICIPACION_EDAD_NOVATO_* (ver sección 9)
TITULAR_PENALIZACION_NOVATO_MAX	0.25	Penalización máxima al peso de titularidad por ser un novato de 16-19 años
PRESTAMO_PESO_TITULAR_UMBRAL / PRESTAMO_PROMEDIO_UMBRAL	0.35 / 6.0	Umbrales de clubDebePrestar — cuándo tu club te cede a préstamo en vez de renovarte o venderte (ver sección 16.9)
PETICION_NUMERO_BASE / _OVR_PESO / _EQUIPO_PESO	0.3 / 0.008 / 0.05	Fórmula de aceptación de cambio de dorsal
PROB_LESION_BASE	0.065	Probabilidad base de lesión por pausa
PROB_LESION_EDAD_INICIO / _INCREMENTO	30 / 0.0027	Desde cuándo y cuánto sube el riesgo de lesión con la edad
PROB_LESION_MAX	0.18	Techo de probabilidad de lesión
PESO_LESION_NIVEL1 / _NIVEL2	0.10 / 0.35	Probabilidad de que, si hay lesión, sea nivel 1 o nivel 2 (nivel 3 es el resto, ~0.55)
LESION_NIVEL3_DURACION	1	Duración fija de una lesión leve (en tramos)
LESION_NIVEL2_DURACION_MIN/MAX	1 / 2	Rango de duración de una lesión moderada

Constante	Valor	Qué controla
LESION_NIVEL1_DURACION_MIN	2	Mínimo de duración de una lesión grave (el máximo es el resto de la temporada)
LESION_NIVEL2_OVR_MIN/MAX	1 / 3	Rango de OVR perdido por una lesión moderada
LESION_NIVEL1_OVR_MIN/MAX	4 / 10	Rango de OVR perdido por una lesión grave
LESION_RECUPERACION_OVR	0.5	% del OVR perdido por lesión que se recupera al darte de alta
CAPITAN_UMBRAL_TEMPORADAS	3	Temporadas mínimas en el mismo club para poder ser nombrado capitán
CAPITAN_UMBRAL_PESO_TITULAR	0.75	pesoTitular mínimo (con el que arranca la próxima temporada) para ser nombrado capitán, ver sección 9
FAMA_MAX	100	Techo de la fama acumulada
FAMA_POR_TROFEO	2	Fama ganada por cada trofeo de club o de selección, al cierre de temporada
FAMA_POR_PREMIO_INDIVIDUAL	6	Fama ganada por cada Bota/Balón de Oro o Once Ideal, al cierre de temporada

25. Limitaciones conocidas y notas para el futuro

- `hayCarreraGuardada()` / `limpiarPartidaGuardada()` ya están implementadas en el store (ver [sección 23](#)) pero ninguna vista las usa todavía — no hay pantalla de "continuar carrera" ni forma de borrar un guardado desde la interfaz, solo recargar `/carrera` (que carga automático) o borrar `localStorage` a mano.
- `pierna hábil` se guarda pero no se usa en ninguna fórmula todavía — es puramente cosmético en la ficha/camiseta.
- Los números de partidos por competición ([sección 21](#)) son una referencia realista, no oficiales fijos, para ligas/copas cuyo formato cambió seguido en la realidad (Argentina, México, Colombia) — están documentados caso por caso en los comentarios de `data/database.ts`.
- 24 de los 47 países de la creación de personaje tienen liga propia cargada (subió de 5 con la incorporación de 19 ligas nuevas); el resto arranca "de extranjero" en las 5 grandes ligas europeas — es coherente con el diseño actual (documentado en [sección 5](#)), no un bug, pero sigue siendo la superficie más obvia para sumar más ligas locales a futuro.
- **Rusia sigue cargada como confederación UEFA**, aunque sus clubes están suspendidos de las competiciones de UEFA desde 2022 — el juego no modela esa suspensión, así que un club ruso con la fuerza suficiente sí puede "clasificar" a la Champions/Europa League en la ficción del juego. Es una simplificación deliberada (no hay ningún mecanismo de "confederación con competiciones restringidas"), no un error de tipeo.
- **La Liga Premier de Ucrania quedó con 16 equipos** en vez de los 20 reales de esta temporada (ver [sección 21](#)) — se prefirió no completarla con clubes inventados sin escudo real.
- **Turquía, Grecia, Rusia, China y Ucrania** tienen liga y equipos cargados pero todavía no son nacionalidades elegibles en la creación de personaje — se puede fichar por sus clubes durante la carrera, pero no arrancarla siendo local ahí.
- **AFC todavía no tiene copa continental de segundo nivel** (a diferencia de UEFA/CONMEBOL) — un club japonés o chino solo puede clasificar a la AFC Champions League Elite, nunca a un equivalente de la Europa League/Sudamericana.
- **CAF no tiene ninguna liga doméstica de club cargada** — solo existe como confederación de selecciones nacionales (para la Copa Africana de Naciones, [sección 19](#)); ningún club africano es fichable todavía.
- **El rival de carrera** ([sección 14.3](#)) sigue generándose y avanzando en cada carrera, pero está oculto en la interfaz a pedido (feedback de testers) — no se muestra en ningún lado hoy. No es un bug ni un olvido: el motor quedó intacto por si se reactiva más adelante.